

慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/05/28)

08 - Transformer を理解する

情報工学科 准教授 高道 慎之介



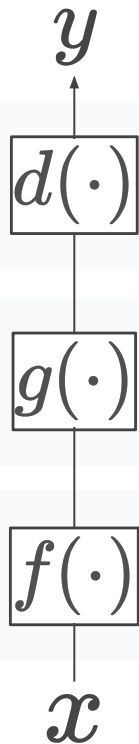
Takamichi Laboratory
慶應義塾大学 高道研究室

講義予定

第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	2026/04/16	自然言語処理のための機械学習に入門する	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	2026/04/30	単語をベクトルで表現する	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習 1	これまでの復習
第08回	2026/05/28	Transformerを理解する	大規模言語モデル時代の技術
第09回	2026/06/11	言語モデルを事前学習する	
第10回	2026/06/18	言語モデルを転移学習する	
第11回	2026/06/25	言語を生成 (デコーディング) する	
第12回	2026/07/02	言語やラベルを評価する	
第13回	2026/07/09	自然言語処理から音声言語処理へ	おまけ
第14回	2026/07/16	演習 2	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：エンコーダ・デコーダ

復習：x から y の予測



決定 (decision) → 本講義では確率値の最も高い候補を選択

- 出力候補のうちから 1 つを決定すること

スコア計算 (score calculation) → 本講義では出力候補の確率を計算

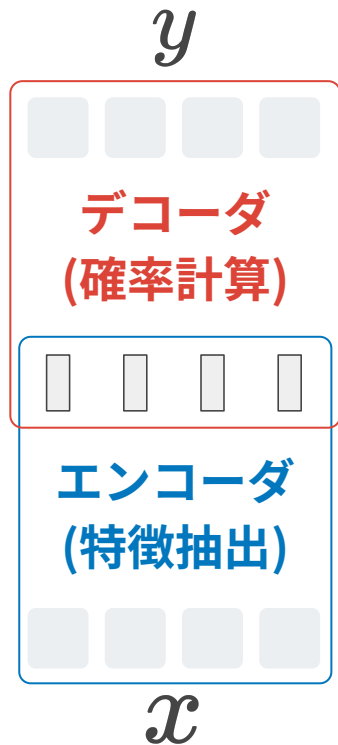
- 特徴に基づいて、出力候補とそのスコアを計算すること

特徴抽出 (feature extraction)

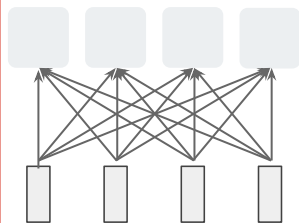
- 意味のある特徴を入力テキストから抽出すること

入力と出力が系列になった場合を考えよう

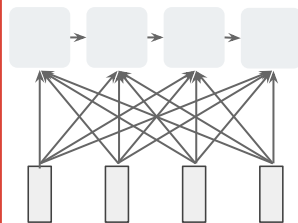
各出力をどの (別) 出力から計算する？



各出力独立

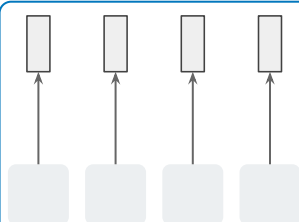


過去出力

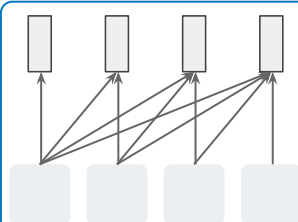


全特徴を使うと
仮定して描写

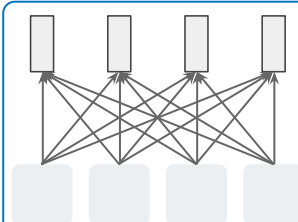
各単語独立



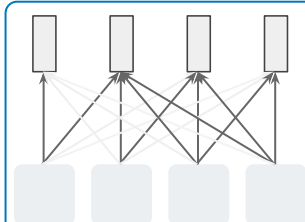
過去単語



全単語



全単語(重み付)



各特徴をどの単語から計算する？

RNN エンコーダ + RNN デコーダ = 系列変換モデル

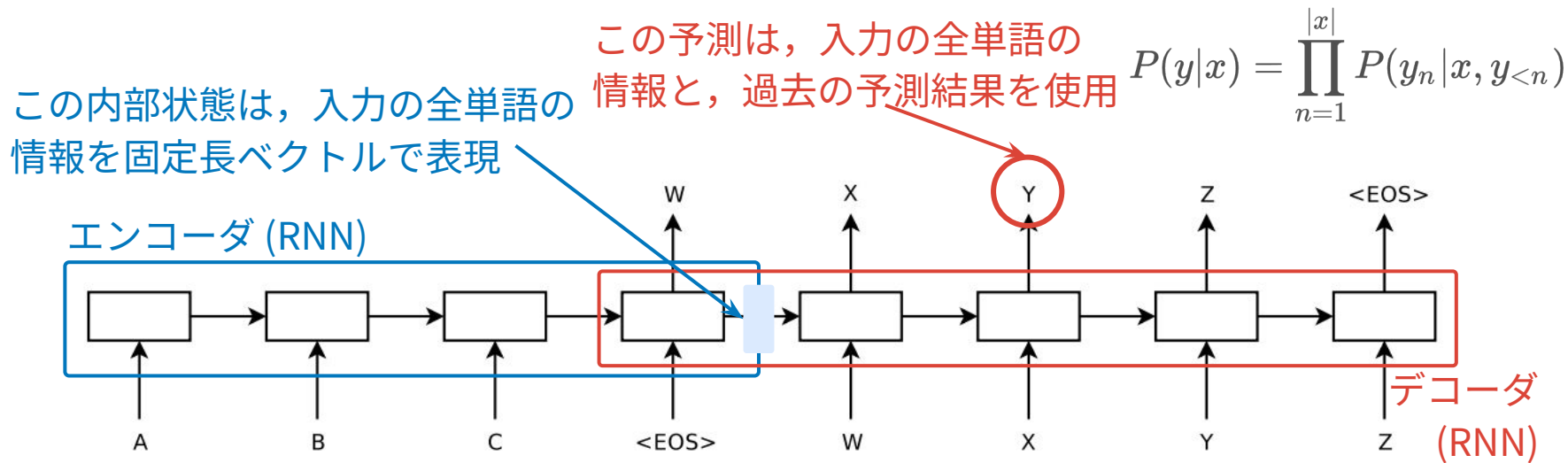


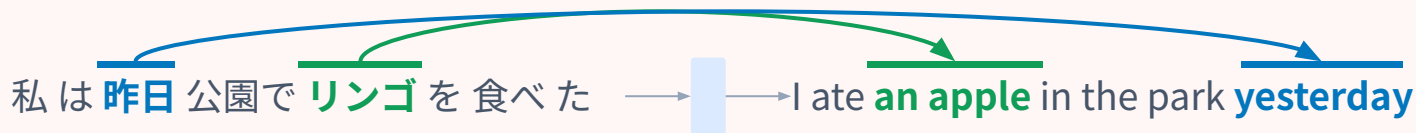
Figure 1: Our model reads an input sentence “ABC” and produces “WXYZ” as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier.

入力と出力の系列長が異なっても変換できる系列変換モデル

RNNの限界と課題

課題 1：全予測で画一的な特徴を使用

予測のタイミングに応じて、対応する単語の情報だけを取り出せない



全単語の情報は、固定長ベクトルで表現される

課題 2：長期依存性の欠如（情報の忘却）

距離が遠い単語間の関係（主語との対応など）を保持できない

彼が昨日ひとりで図書館に行って借りてきた本

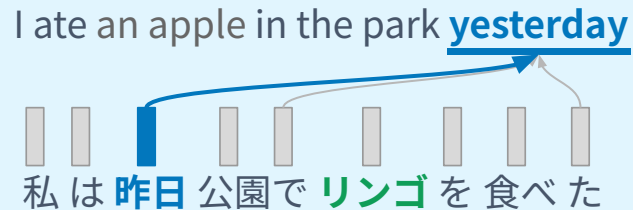
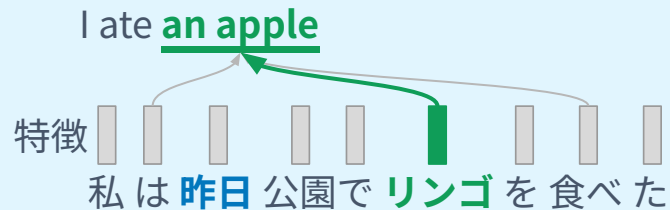


情報の伝播とともに、初期の情報（「彼」など）が薄れてい

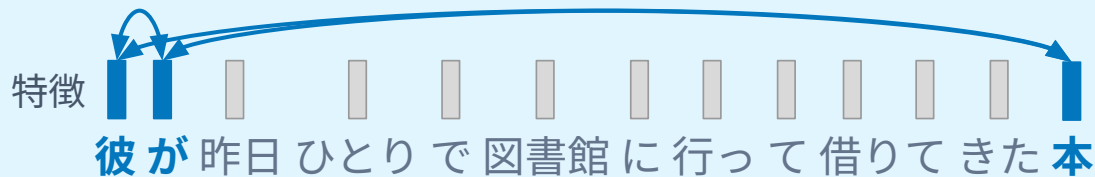
く

モチベーション

モチベ1：必要に応じて特徴ベクトルを取り出せたらよさそう！



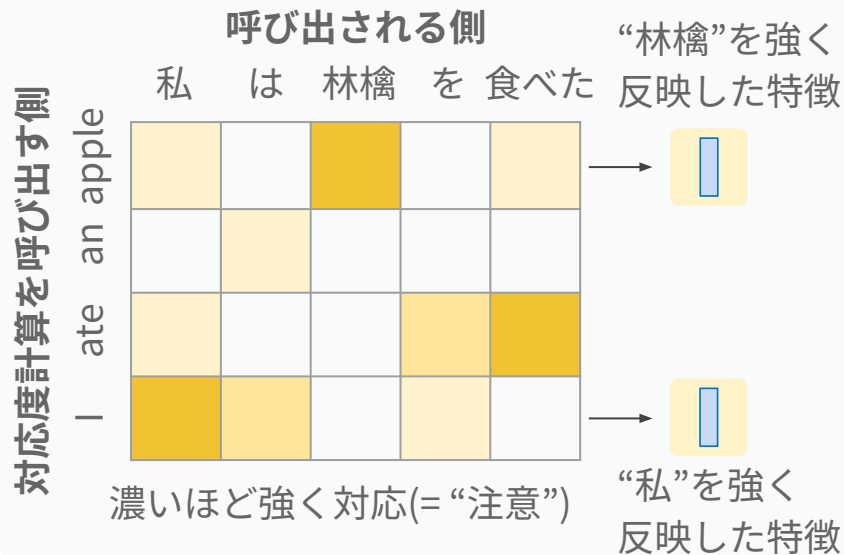
モチベ2：位置に関係なく「関連していること」を特徴づけられたらよさそう！



注意機構の基本的な考え方

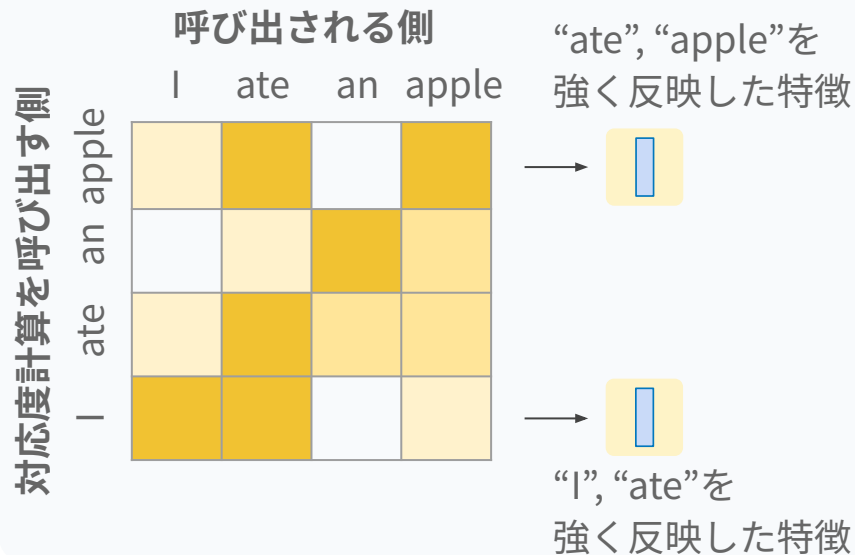
Cross attention

異なる 2 系列 (呼び出す側, 呼び出される側) 同士で関連度を計算．呼び出す側に必要な特徴を随時取り出す．



Self attention

同じ系列 で関連度を計算．特徴同士の関連を計算するため，意味の有る特徴を計算できる．



出力側がクエリ，入力側がキーになる

I ate

Decoder (e.g., RNNs)



クエリ

q_2

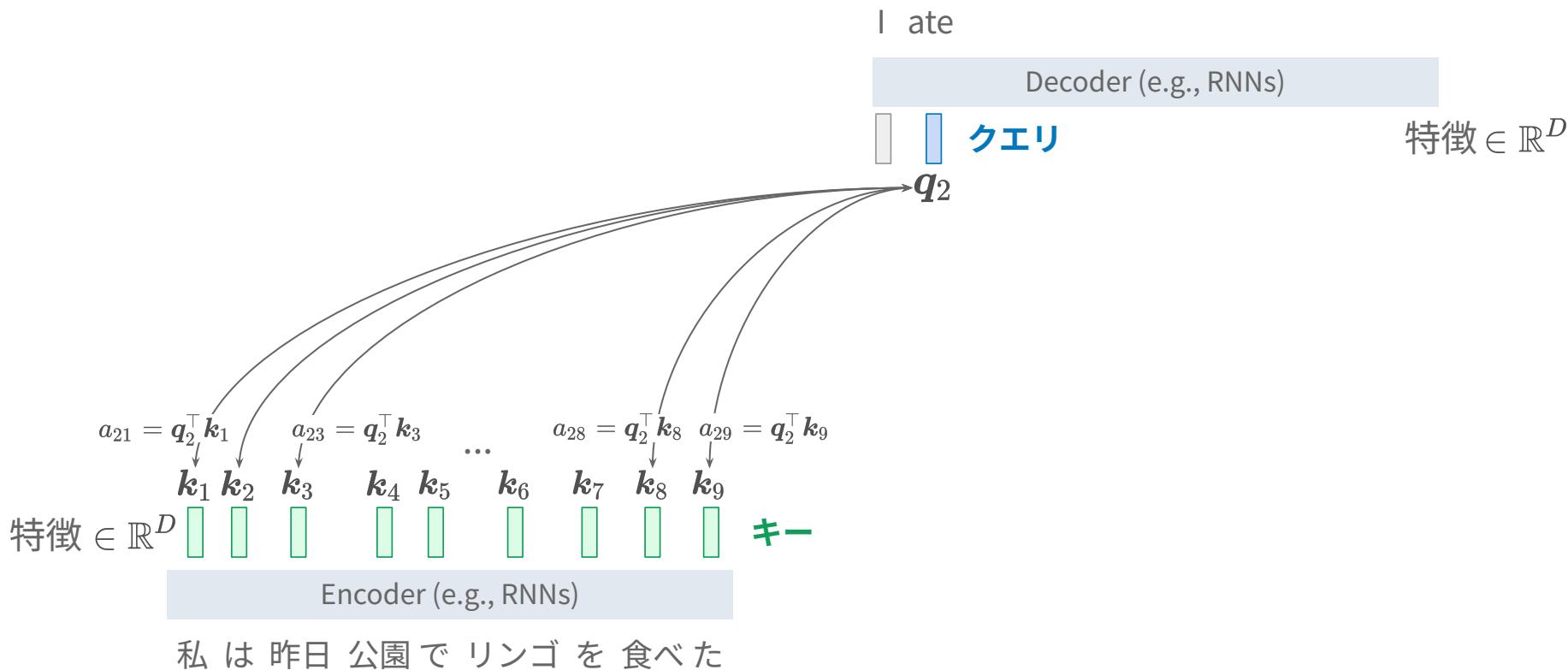
特徴 $\in \mathbb{R}^D$

特徴 $\in \mathbb{R}^D$ k_1 k_2 k_3 k_4 k_5 k_6 k_7 k_8 k_9 キー

Encoder (e.g., RNNs)

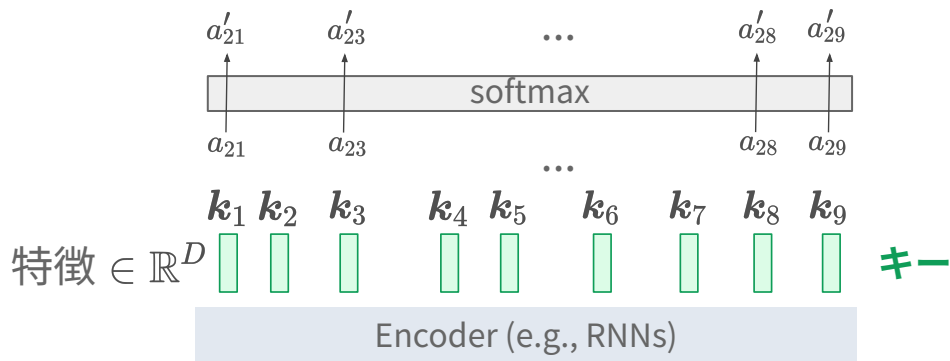
私は昨日公園でリンゴを食べた

クエリとキーの内積をとり対応度を求める



Softmax をかけて正規化 (attention の値にする)

各単語がクエリの単語に
どれくらい関連しているかを表す



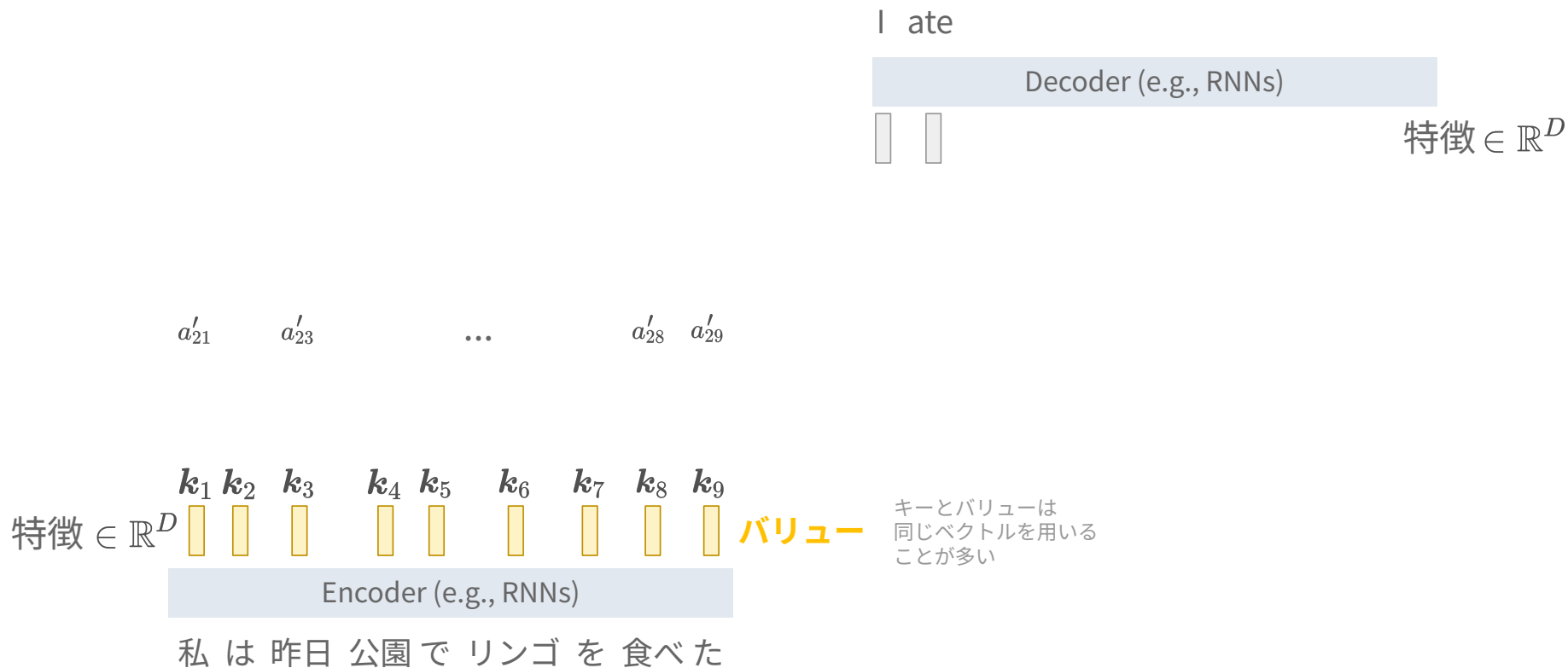
I ate

Decoder (e.g., RNNs)

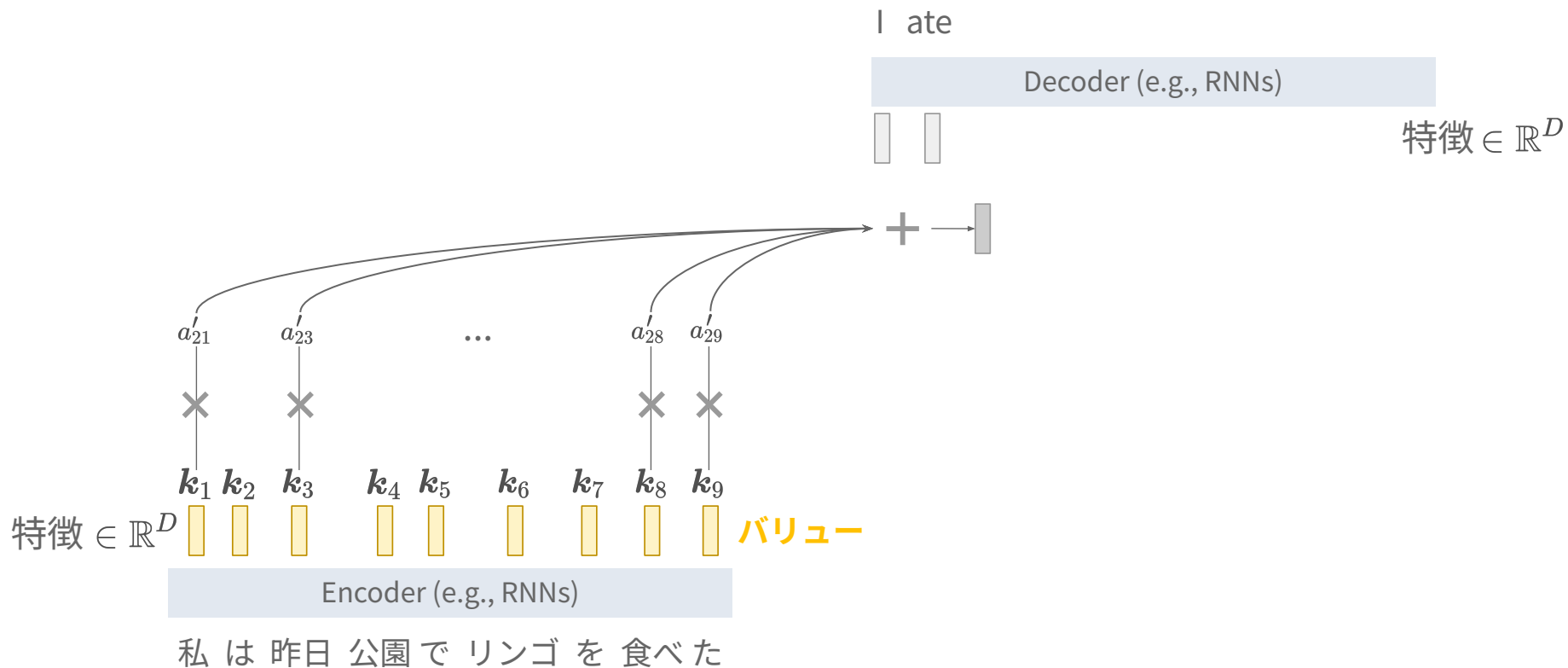


特徴 $\in \mathbb{R}^D$

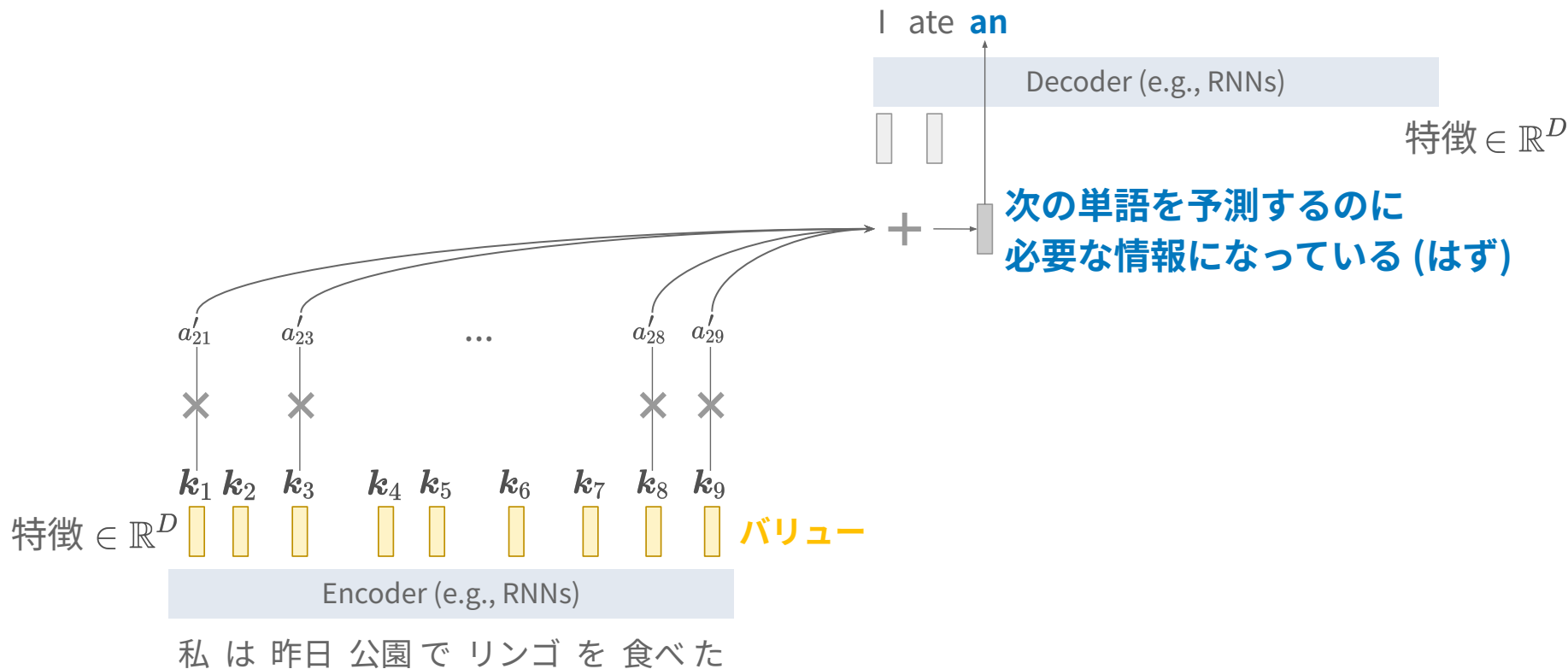
検索される側にバリューを用意する



バリューを attention 値で重みづけて足し合わせ



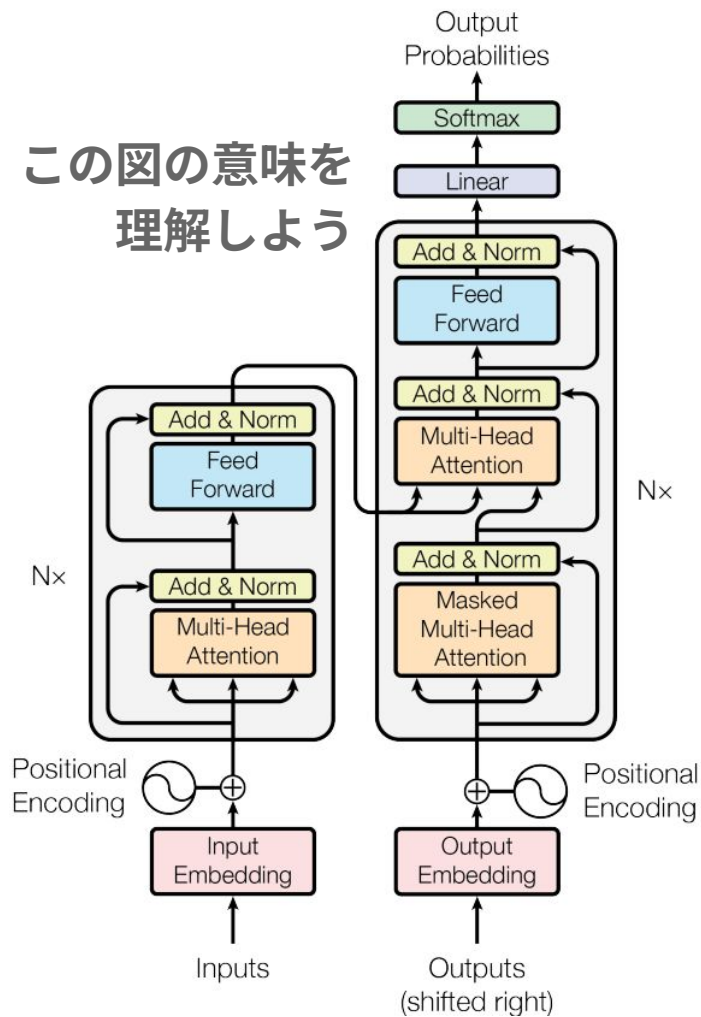
バリューを attention 値で重みづけて足し合わせ



今日の目的：

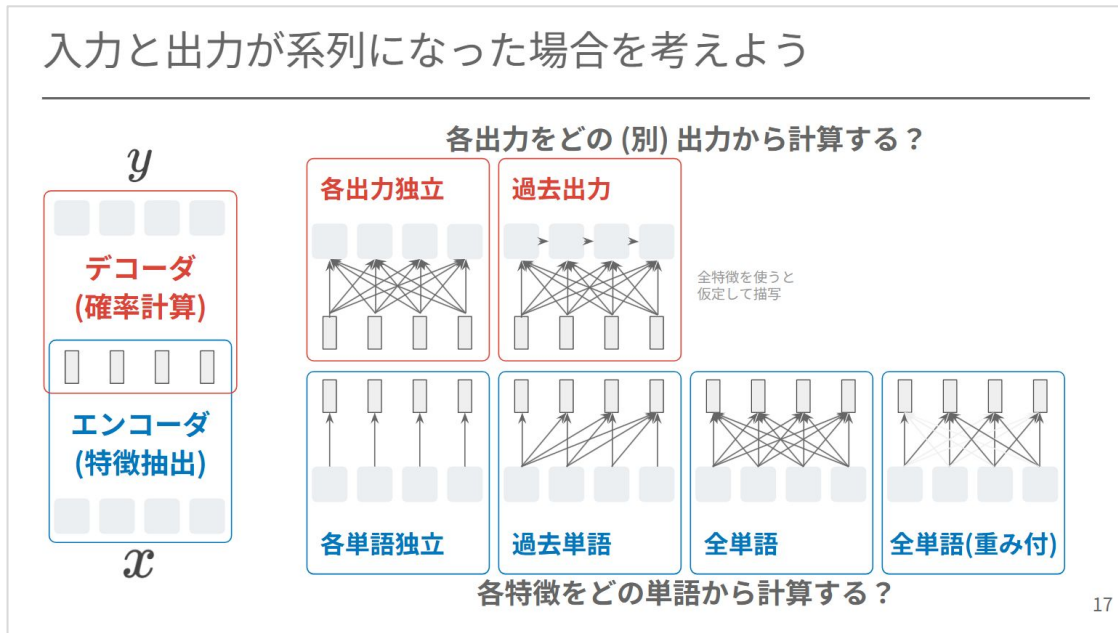
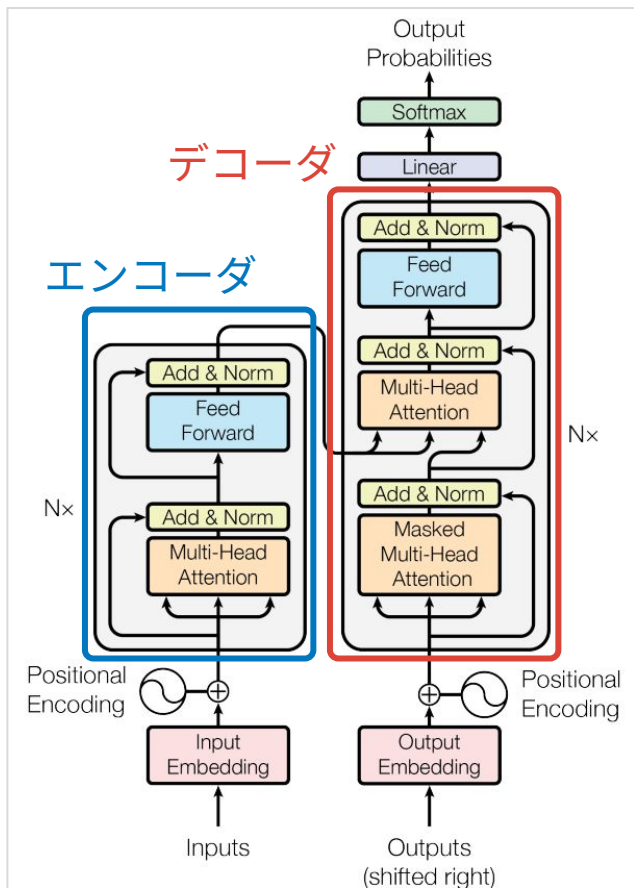
Transformer & GPT の構成要素を図と式で理解する

この図の意味を
理解しよう



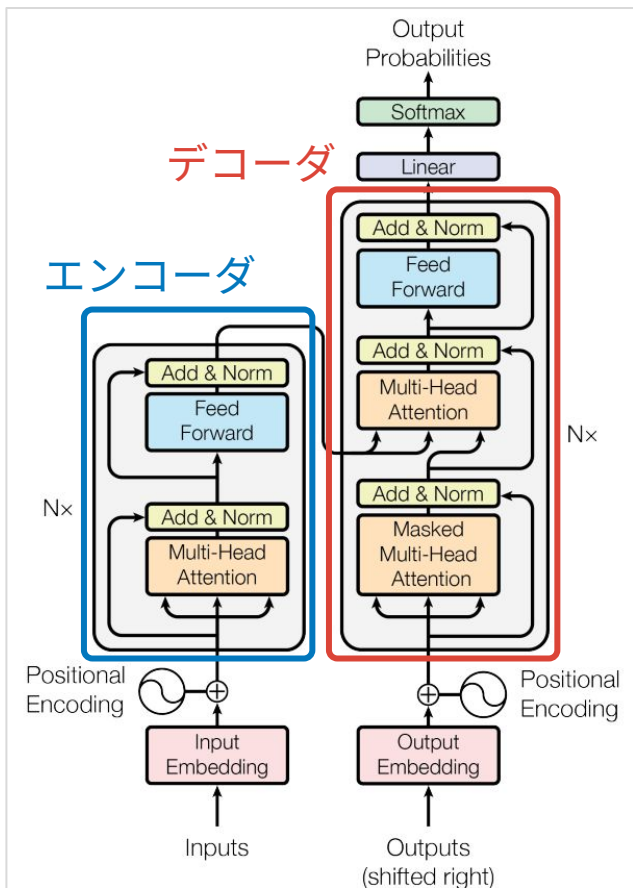
まずは全体像を理解する

(1) Transformer はエンコーダとデコーダから成る



第 06 回スライドから引用

(2) Transformer は系列変換モデルの一種である



エンコーダ：各単語の特徴を抽出

デコーダ：過去の出力を入力に戻して次を予測

RNN エンコーダ + RNN デコーダ = 系列変換モデル

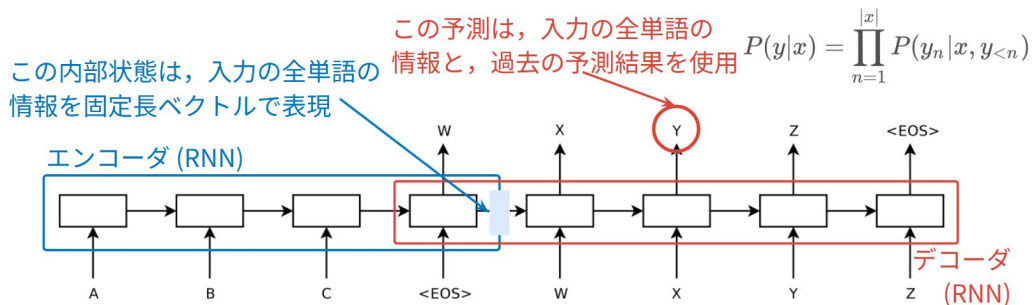
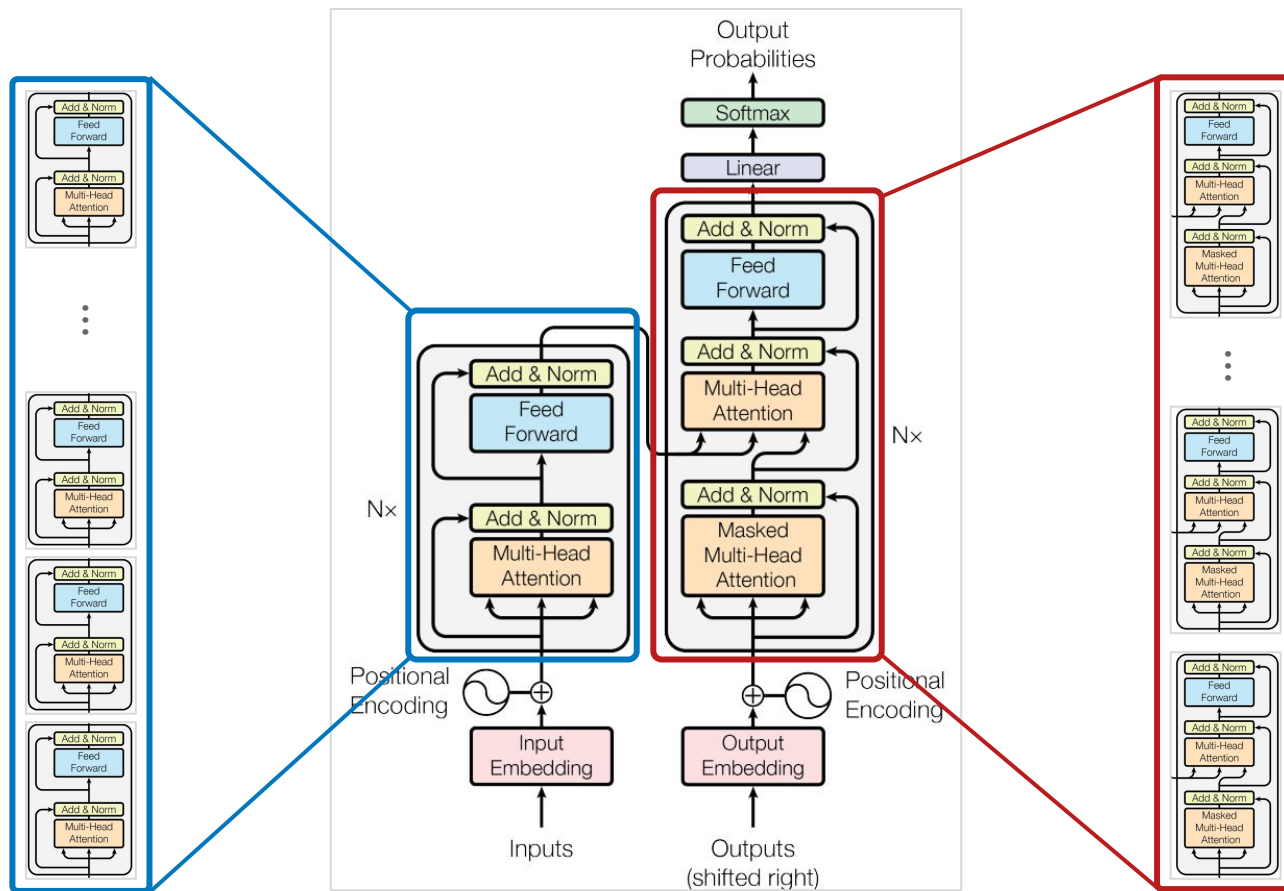


Figure 1: Our model reads an input sentence “ABC” and produces “WXYZ” as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier.

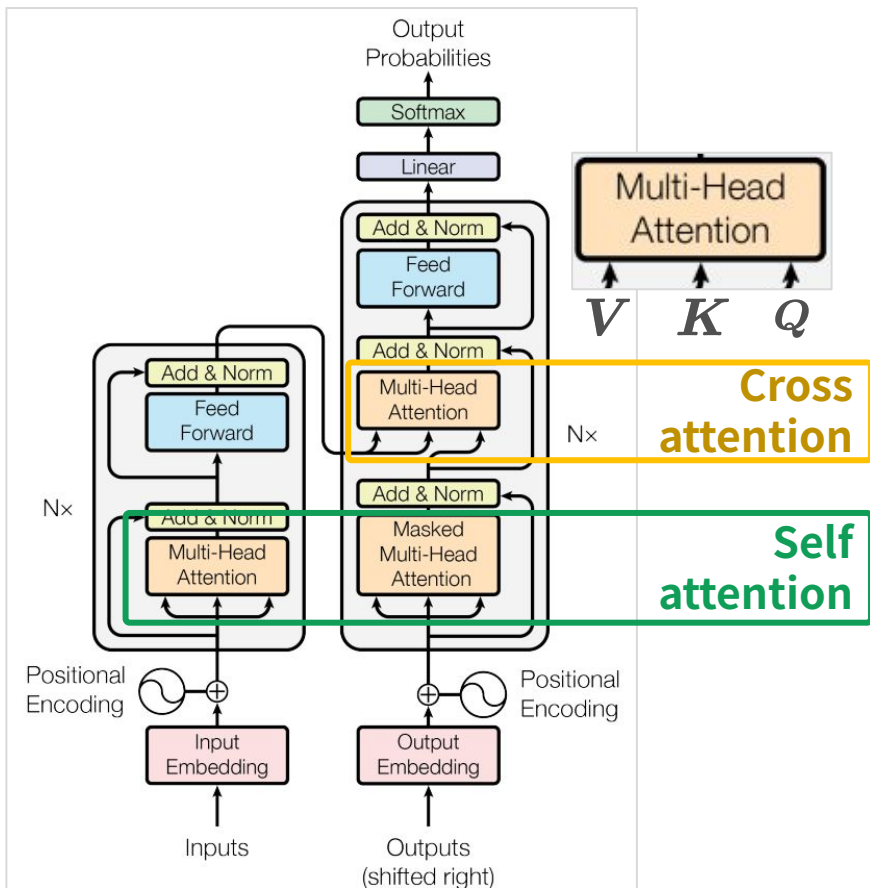
入力と出力の系列長が異なっても変換できる系列変換モデル

第 06 回スライドから引用

(3) Transformer は層のスタックでできている



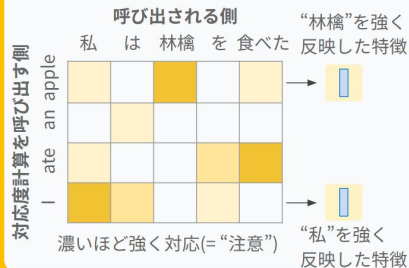
(4) エンコーダもデコーダも注意機構に基づいている



注意機構の基本的な考え方

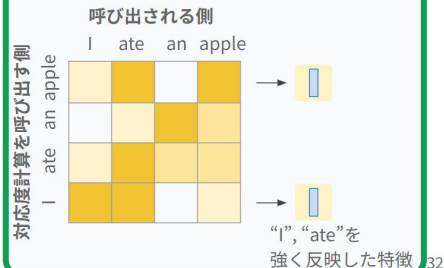
Cross attention

異なる2系列(呼び出す側, 呼び出される側)同士で関連度を計算. 呼び出す側に必要な特徴を随時取り出す.



Self attention

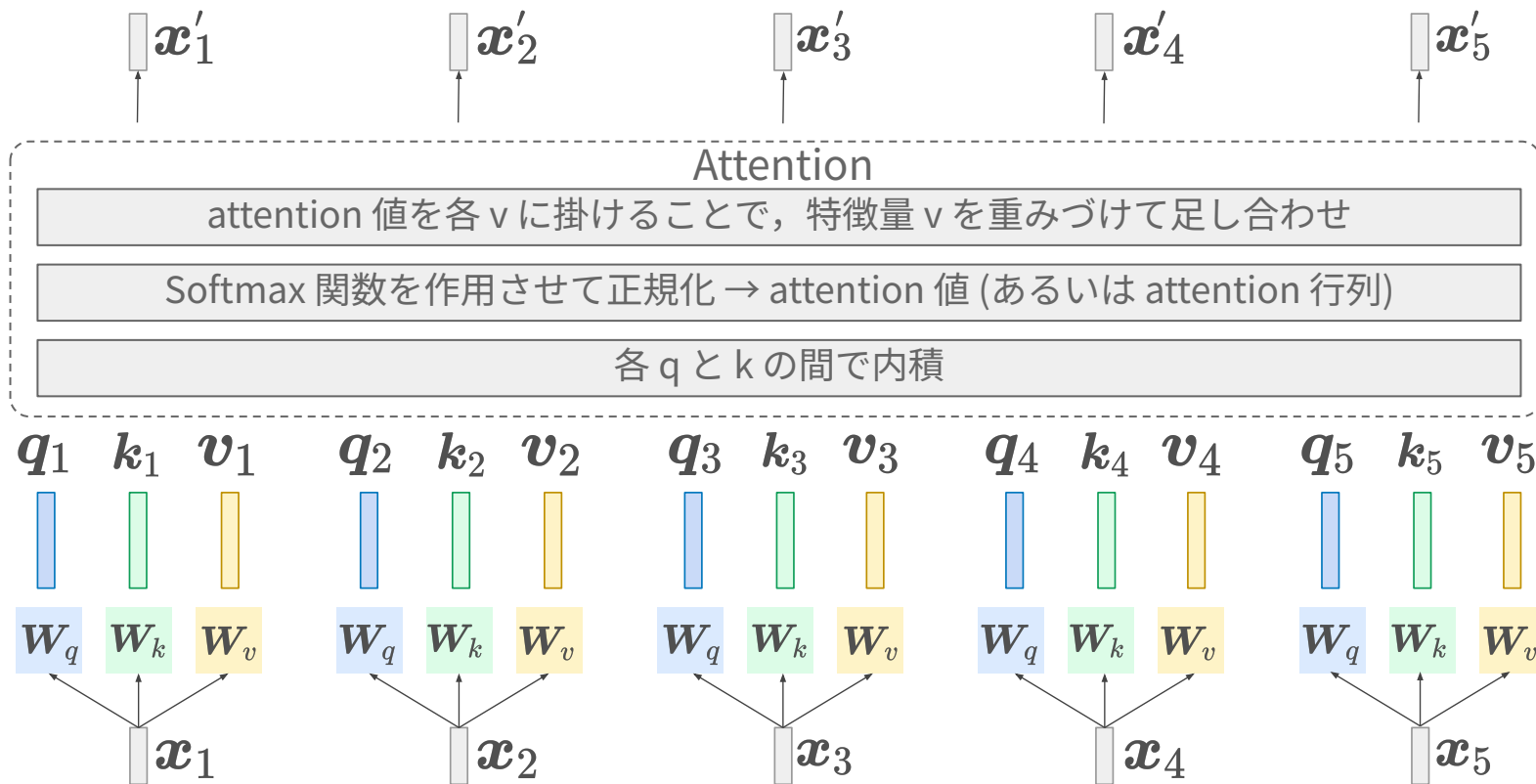
同じ系列で関連度を計算. 特徴同士の関連を計算するため, 意味の有る特徴を計算できる.



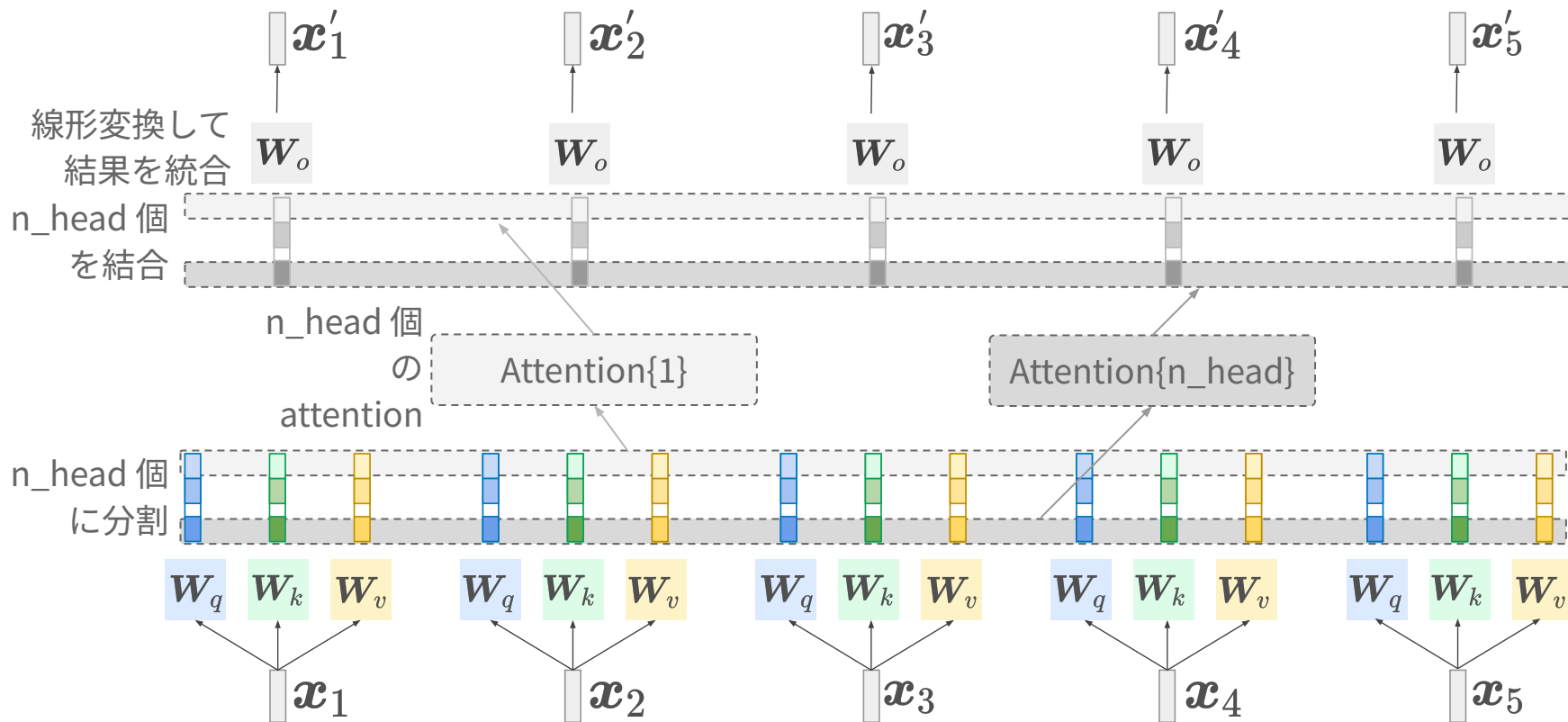
第06回スライドから引用

Transformer のパーツを理解する

復習：注意機構 (詳細は前回の資料を参考にする)

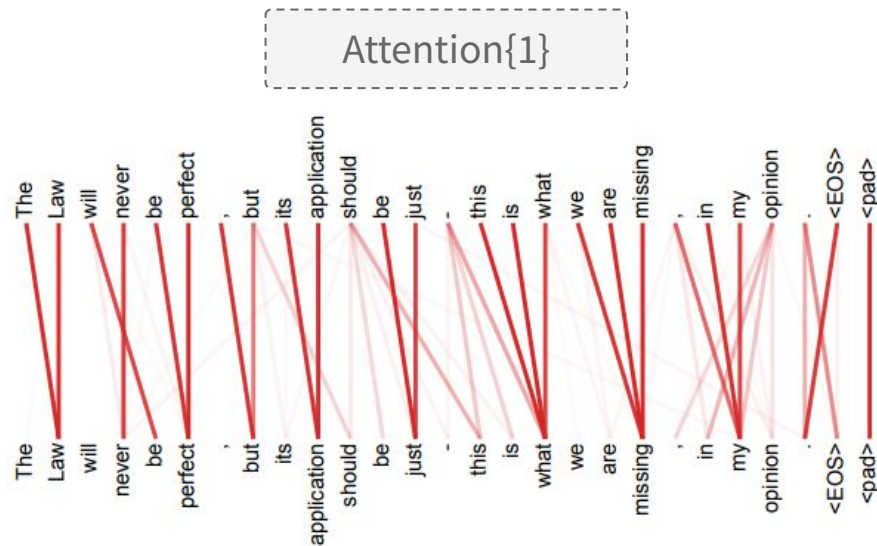


Multi-head 注意機構：q, k, v を分割して注意することで異なる特徴抽出を可能に

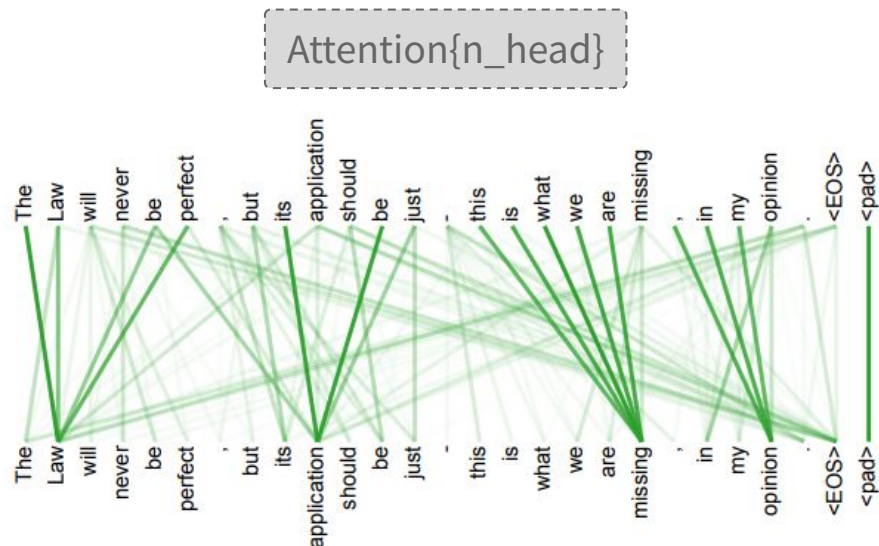


異なる attention head は異なる attention をとる

(後述する Transformer の例)



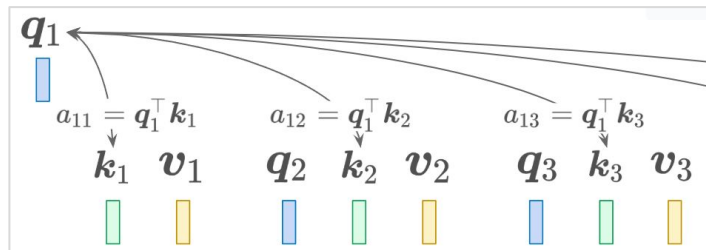
直前の N 単語に attention があたる傾向



遠い位置の単語にも attention があたる傾向

Scaled dot-product attention

$$\begin{array}{c} \mathbf{X}' \\ \begin{bmatrix} \mathbf{x}'_1 \\ \vdots \\ \mathbf{x}'_{|x|} \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{array} = \underset{\substack{\text{softmax}_{\text{row}} \\ \text{QKT} を行ごとに \\ \text{確率値に}}}{\left(\begin{array}{c} \mathbf{Q} \\ \begin{bmatrix} \mathbf{q}_1 \\ \vdots \\ \mathbf{q}_{|x|} \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{array} \begin{array}{c} \mathbf{K}^\top \\ [\mathbf{k}_1, \dots, \mathbf{k}_{|x|}] / \sqrt{d} \\ \mathbb{R}^{d \times |x|} \end{array} \right) \begin{array}{c} \mathbf{V} \\ \begin{bmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_{|x|} \end{bmatrix} \\ \mathbb{R}^{|x| \times d} \end{array} \right. \\ \text{\textcolor{blue}{\sqrt{d}} を導入}$$



ベクトルの内積は次元が高くなるほど値が増加

- softmax を作用させると、特定の要素のみが 1，それ以外が 0 になりがち
- 0 の要素には backpropagation における勾配が消失 → 学習が行われない

softmax の前に、次元数のルート値を除算して緩和

Masked attention

通常の attention

呼び出す側 Q に対し、全要素から K, V を呼び出す。

呼び出される側 K, V

I ate an apple

呼び出す側 Q

I ate an apple

(Causal) masked attention

呼び出す側 Q に対し、それより未来の要素からは呼び出さない (マスクする)
→ 自己回帰的な生成に利用できる

呼び出される側 K, V

I ate an apple

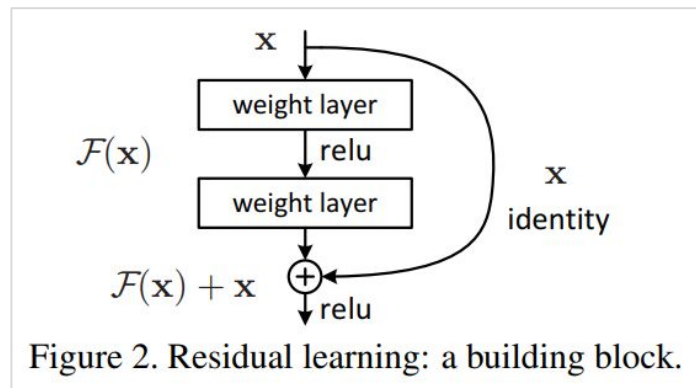
呼び出す側 Q

I ate an apple

			[MASK]
		[MASK]	[MASK]
	[MASK]	[MASK]	[MASK]

Attention の計算から除外
(0 を掛けて無効化する)

残差接続 (residual connection)



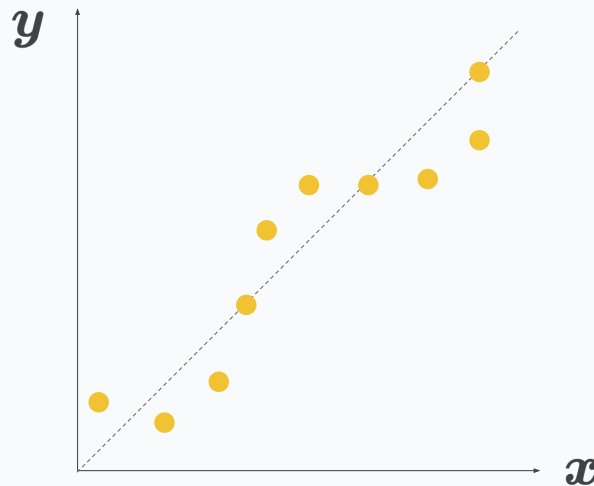
<https://arxiv.org/abs/1512.03385>

通常の変換

$$y = f(x) \in \mathbb{R}^D$$

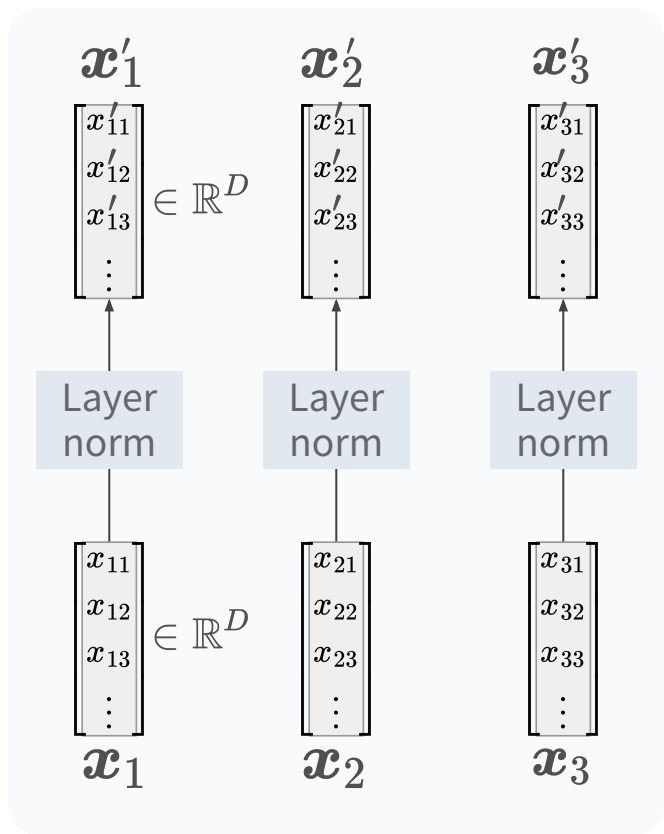
変換 with 残差接続

$$y = f(x) + x$$



- 通常：x と y の複雑な回帰を $f(x)$ に任せる
 - 残差接続： $y = x$ からの差分のみを $f(x)$ で表す
 - 対角線からの差分のみを表せば良い。
 - $f()$ を通らずとも勾配が伝わる。
- モデルを“深層”にできた重要な技術

Layer normalization



特徴量が次元方向に平均 0，分散 1 に正規化

学習時と評価時で分布が変わること (共変量シフト) を緩和できるため，評価データに対して頑健に動作

Transformer においては，単語ごとのスケールを揃える意味がある．例えば，極端に大きな値を持つ単語が有ると，注意機構などがその単語だけに影響される

$$\mu_i = \frac{1}{D} \sum_{d=1}^D x_{id}, \sigma_i = \sqrt{\frac{1}{H} \sum_{d=1}^D (x_{id} - \mu_i)^2}, x'_{id} = \frac{1}{\sigma_i} (x_{id} - \mu_i)$$

i は位置 index

(Sinusoidal) position encoding

Transformer は位置に関する情報を持たないので、位置に関する情報を付与しよう

位置 index を加算

× 大きさが爆発して
ベクトルノルムも増大

位置 index ベクトル

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \end{bmatrix} \quad \begin{bmatrix} 2 \\ 2 \\ 2 \\ \vdots \end{bmatrix} \quad \begin{bmatrix} 3 \\ 3 \\ 3 \\ \vdots \end{bmatrix}$$

$$\begin{matrix} + & + & + \\ \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ \vdots \end{bmatrix} & \begin{bmatrix} x_{21} \\ x_{22} \\ x_{23} \\ \vdots \end{bmatrix} & \begin{bmatrix} x_{31} \\ x_{32} \\ x_{33} \\ \vdots \end{bmatrix} \end{matrix} \in \mathbb{R}^D$$

位置 index を三角関数に分解して加算

- ✓ 値が ± 1 に収まる．周期関数なので長文でも一定範囲に．
- ✓ 相対的な位置変化を回転として表せる

位置を回転位置に変換したベクトル

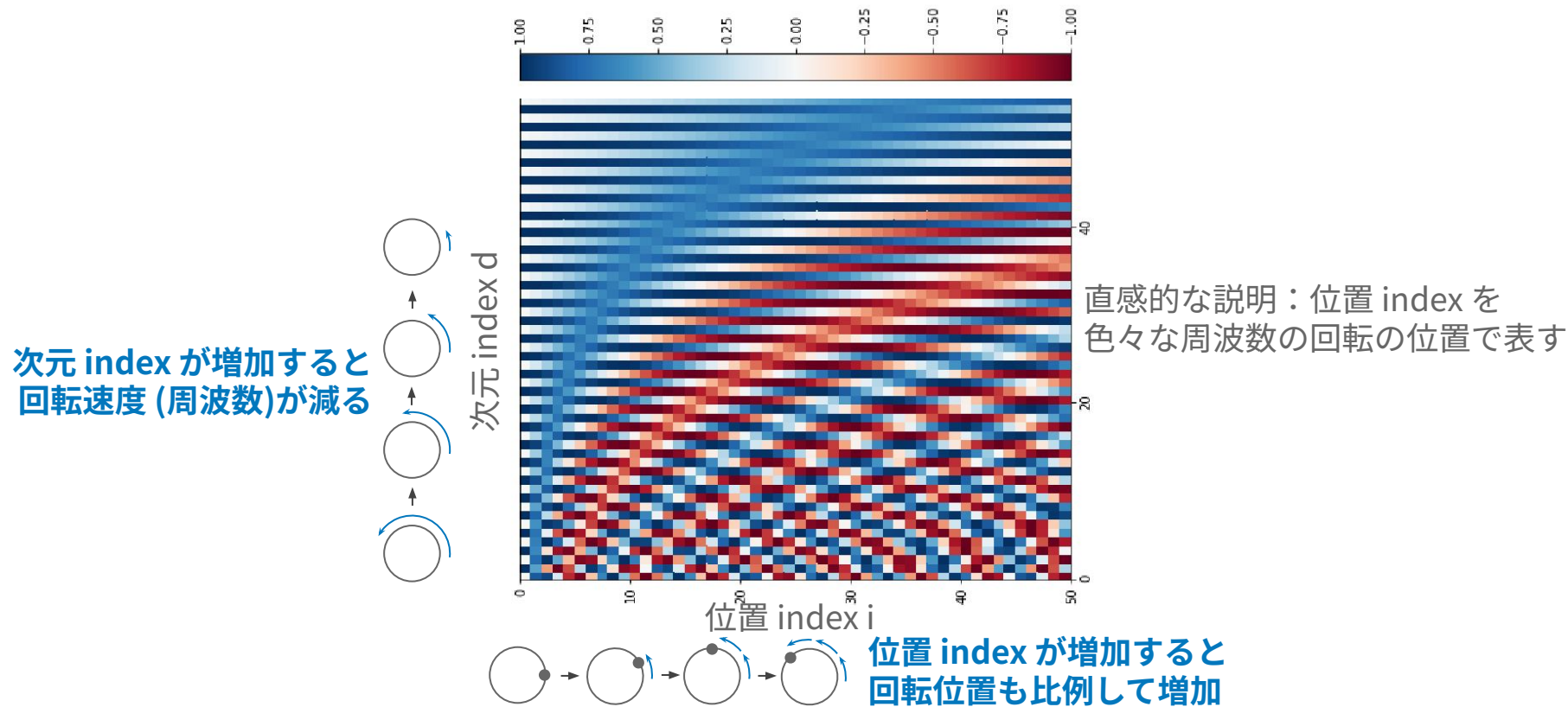
$$t_{id} = \begin{cases} \sin \left(\frac{i}{10000^{2d/D}} \right) \\ \cos \left(\frac{i}{10000^{(2d+1)/D}} \right) \end{cases}$$

i は位置 index, d は次元 index

回転位置
 $\sin \left(2\pi \frac{i}{T} \right)$
回転周期.
 d が増大 $\rightarrow T$ が増大

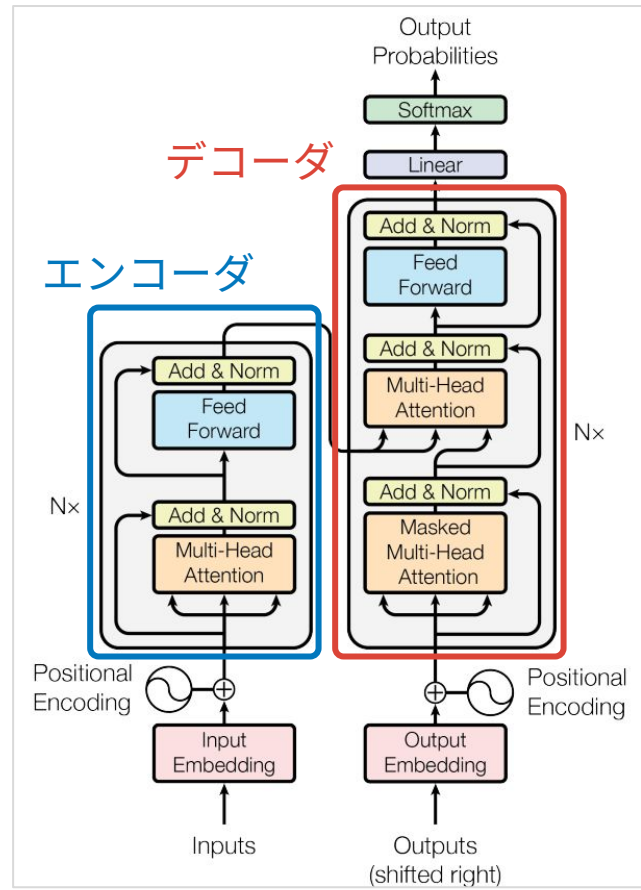
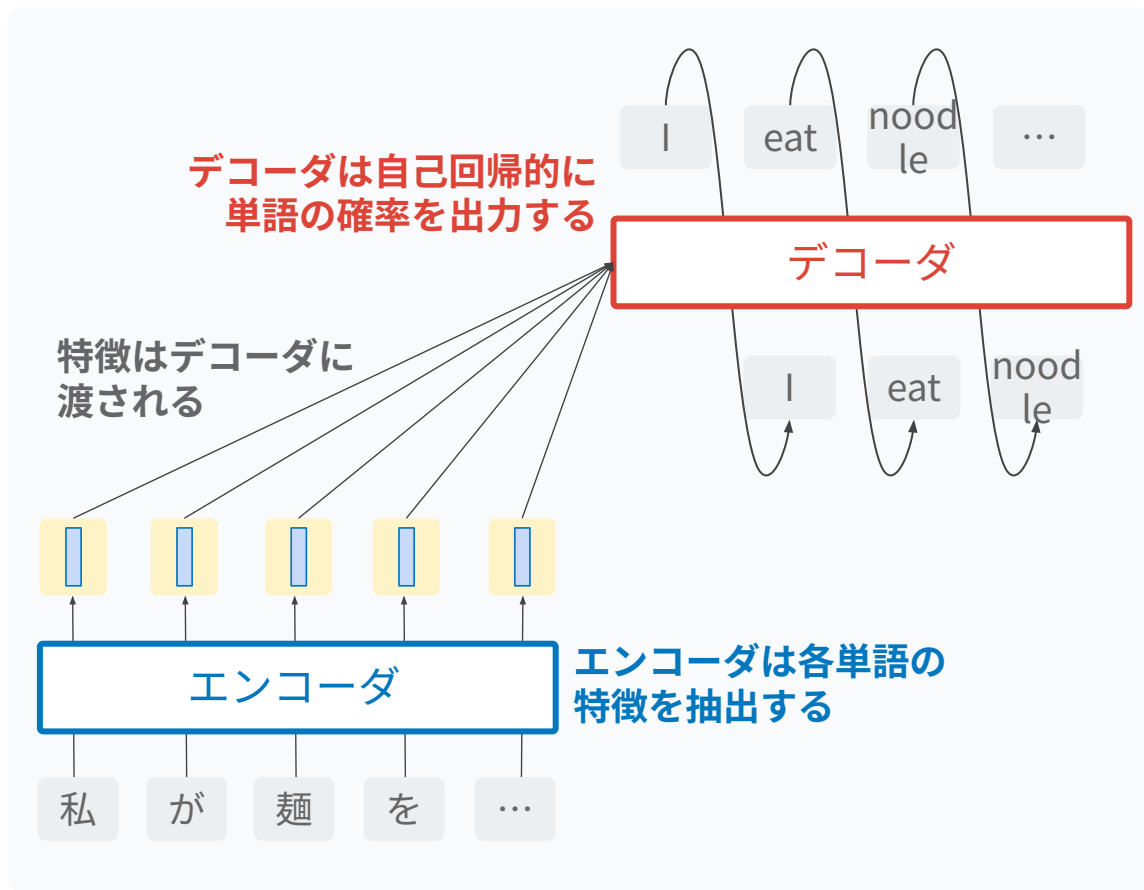
$$\begin{matrix} + & + & + \\ \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ \vdots \end{bmatrix} & \begin{bmatrix} x_{21} \\ x_{22} \\ x_{23} \\ \vdots \end{bmatrix} & \begin{bmatrix} x_{31} \\ x_{32} \\ x_{33} \\ \vdots \end{bmatrix} \end{matrix}$$

(Sinusoidal) position encoding

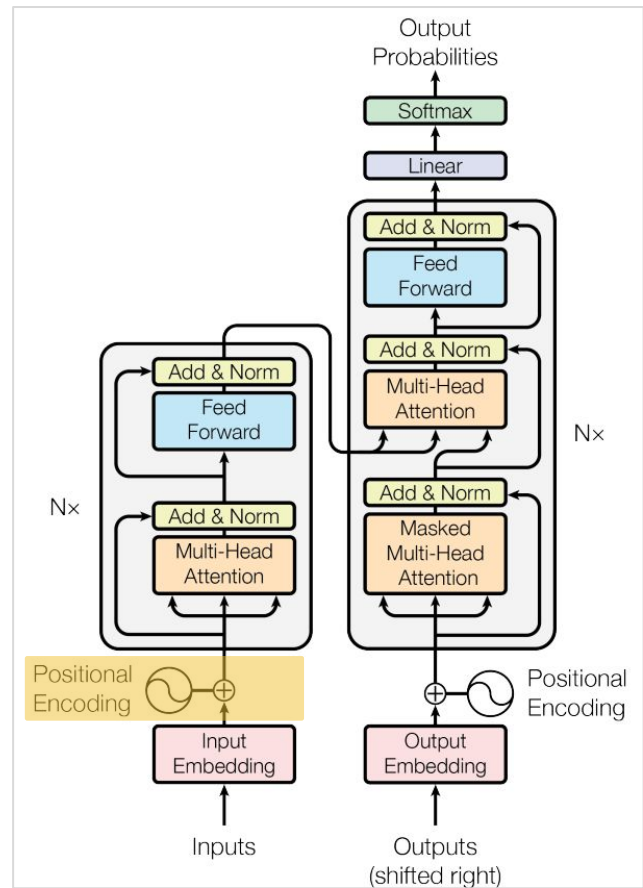
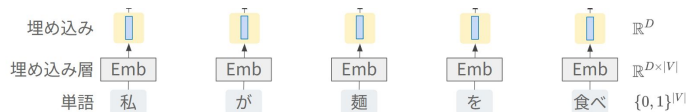


パーツを組み合わせて Transformer を理解する

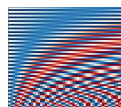
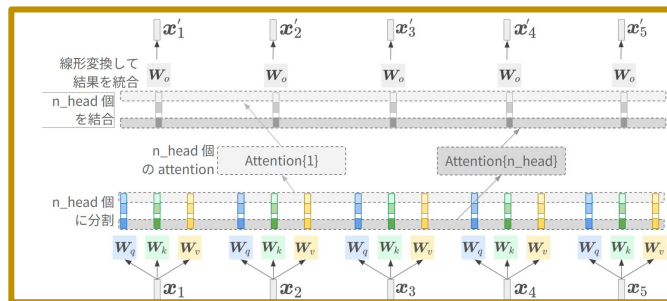
エンコーダとデコーダの役割



エンコーダ②：positional encoding で位置情報を付与



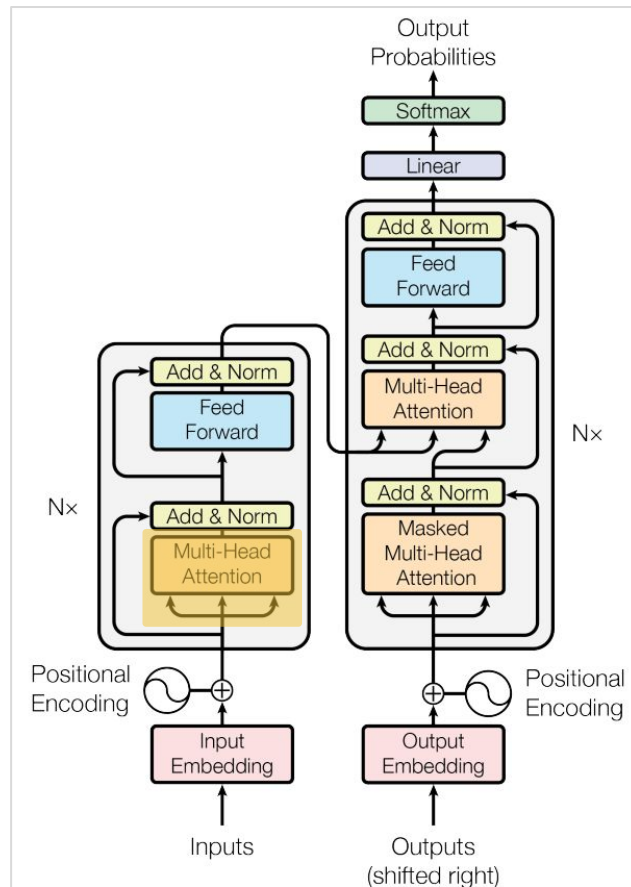
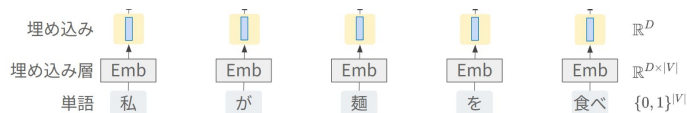
エンコーダ③：multi-head attention で単語関係を学習



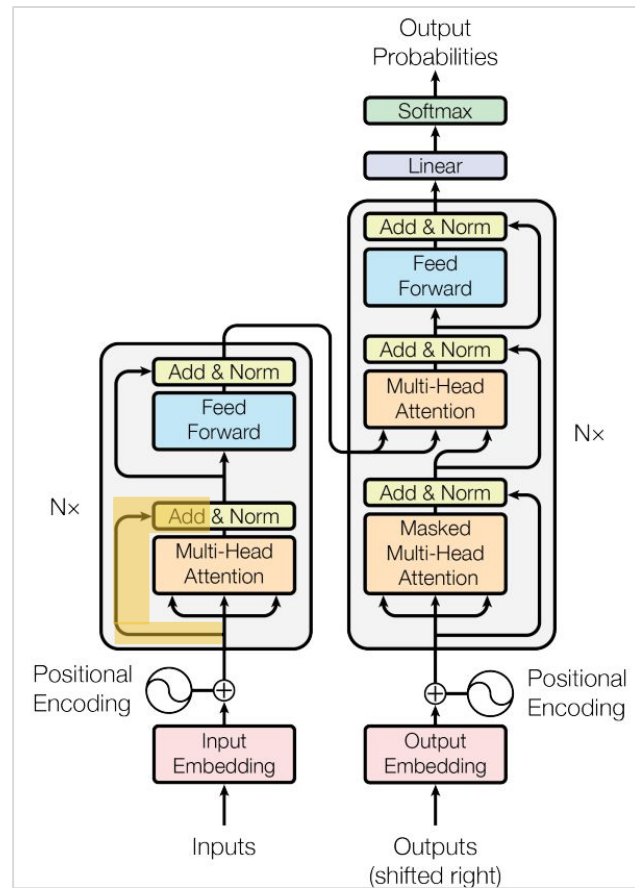
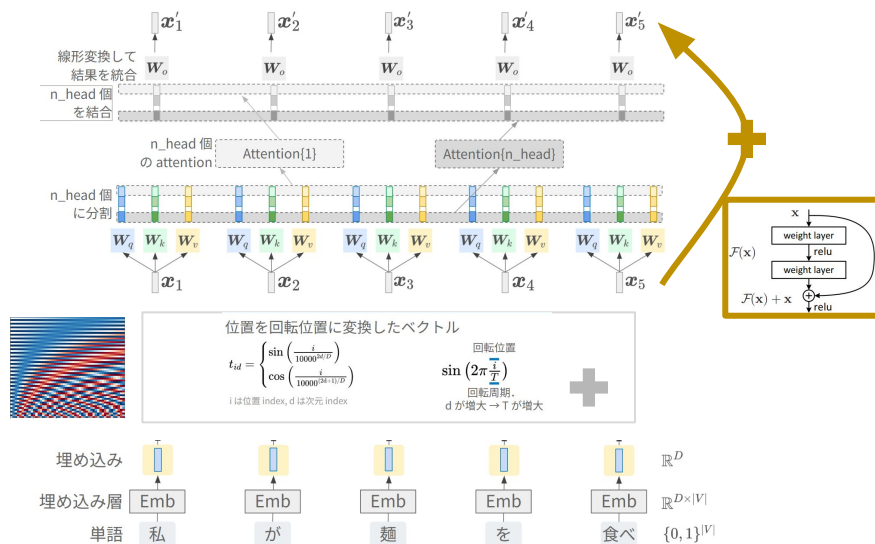
位置を回転位置に変換したベクトル

$$t_{id} = \begin{cases} \sin\left(\frac{i}{10000^{2d/T}}\right) & \text{回転位置} \\ \cos\left(\frac{i}{10000^{2d/T}}\right) & \text{回転周期} \end{cases}$$

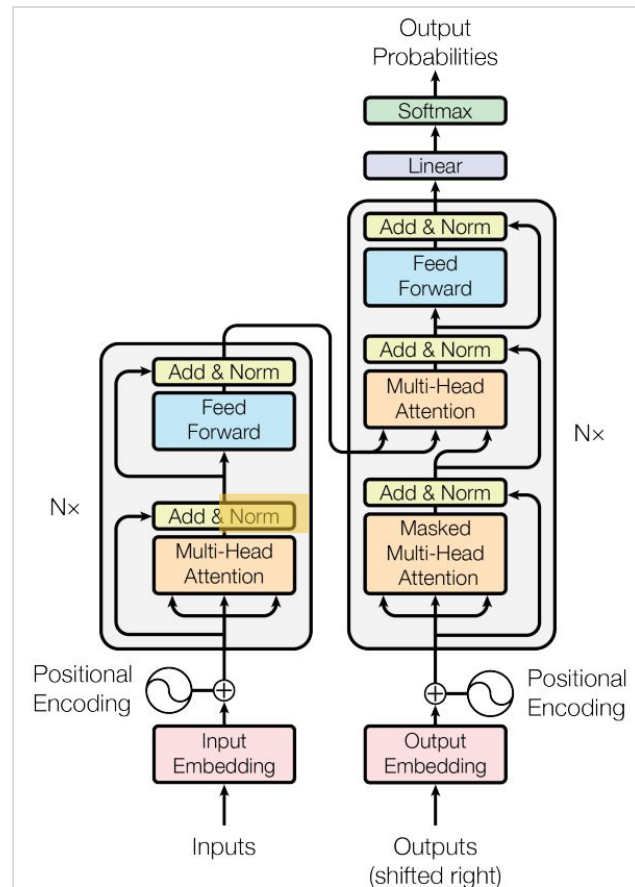
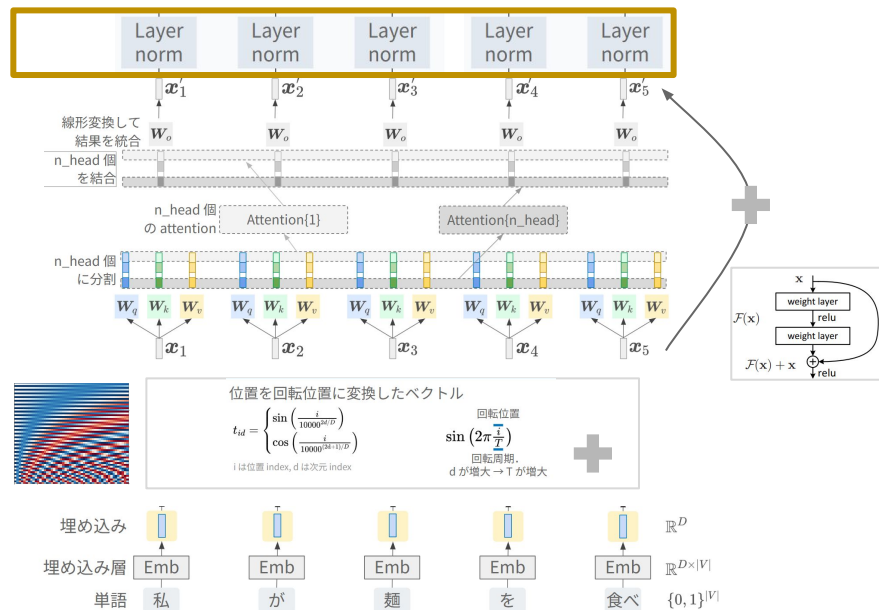
i は位置 index, d は次元 index
 d が増大 $\rightarrow T$ が増大



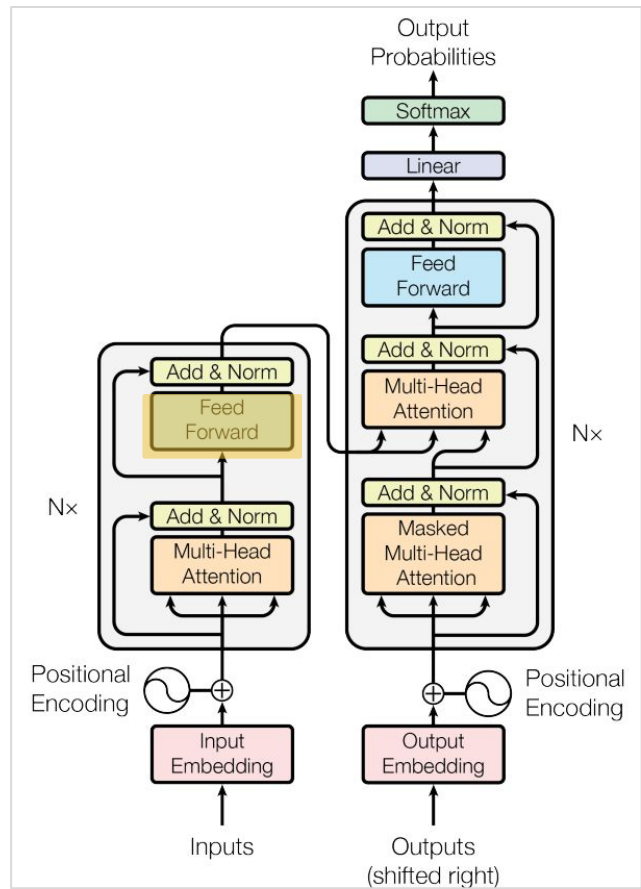
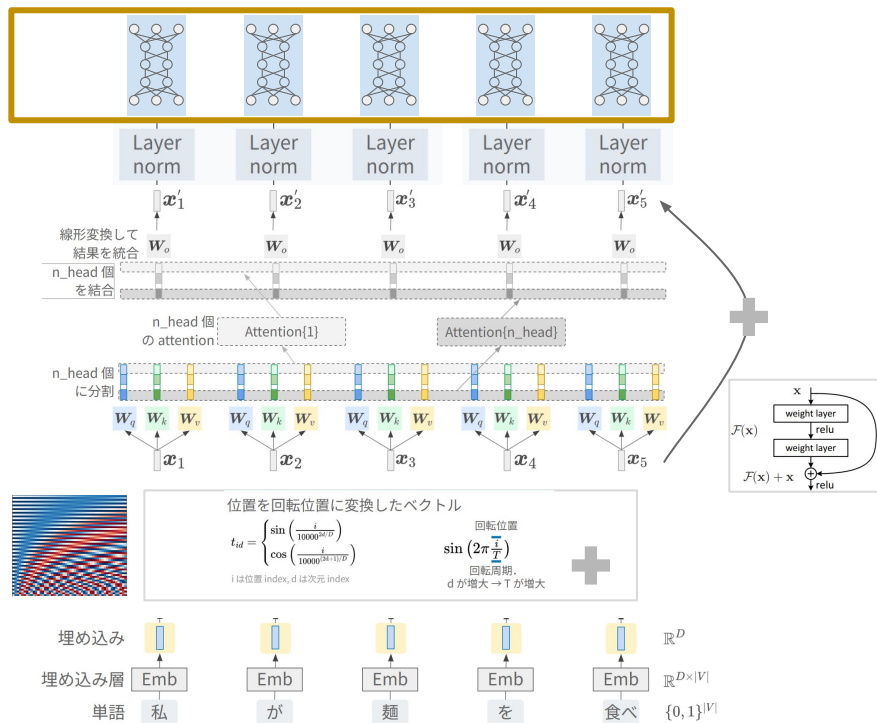
エンコーダ④：residual connection でショートカット



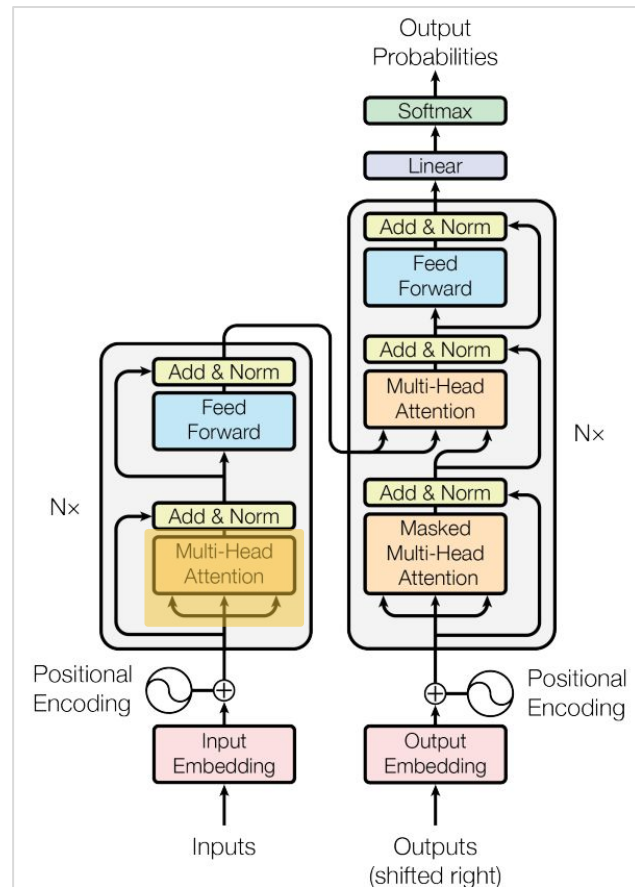
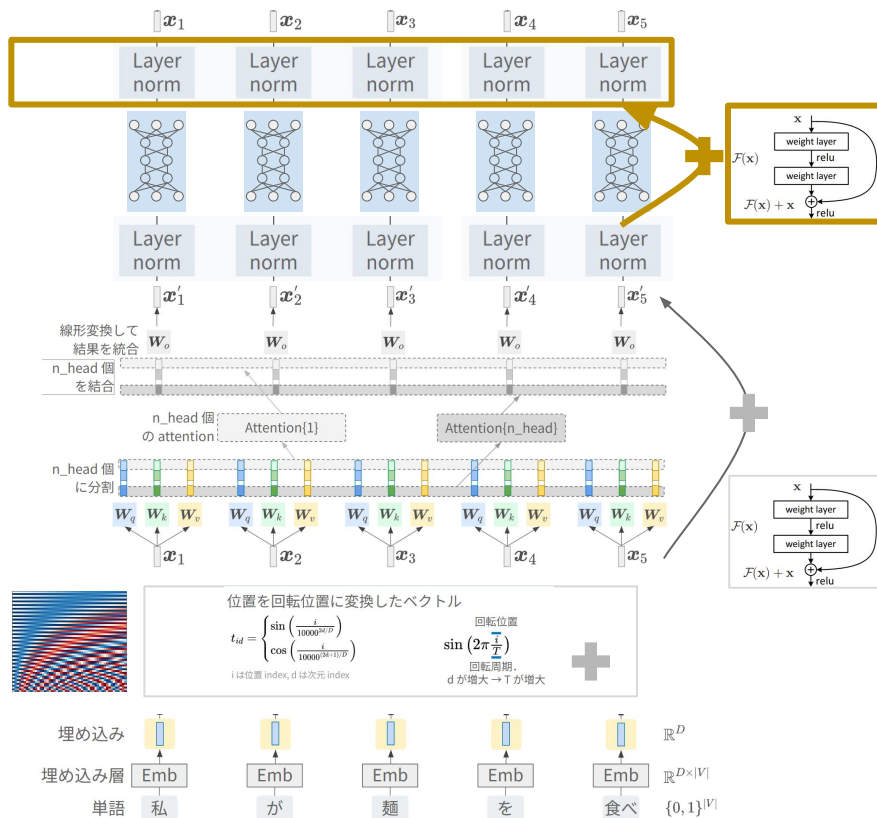
エンコーダ⑤：layer normalization で正規化



エンコーダ⑥：feed forward



エンコーダ⑦：res. connection and layer norm. (again)

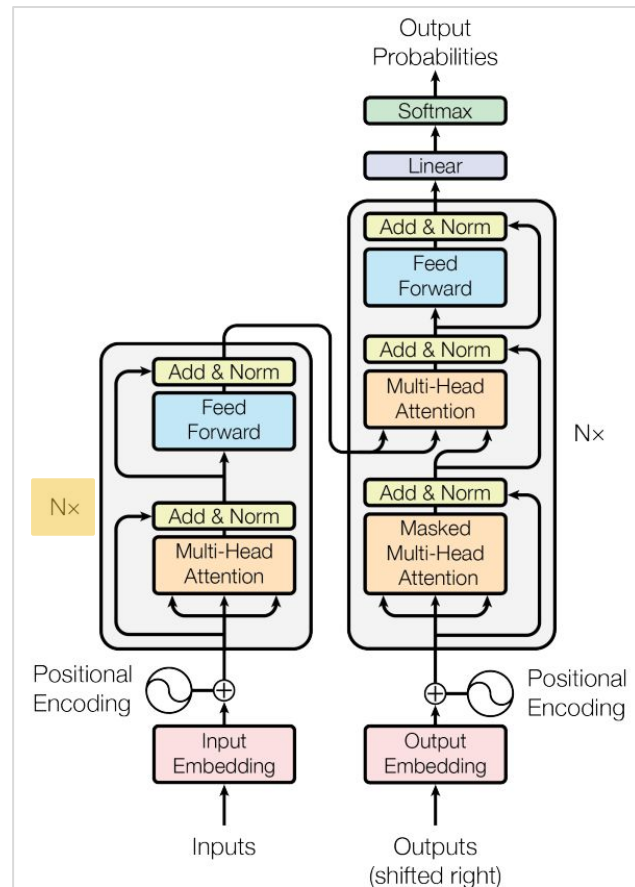
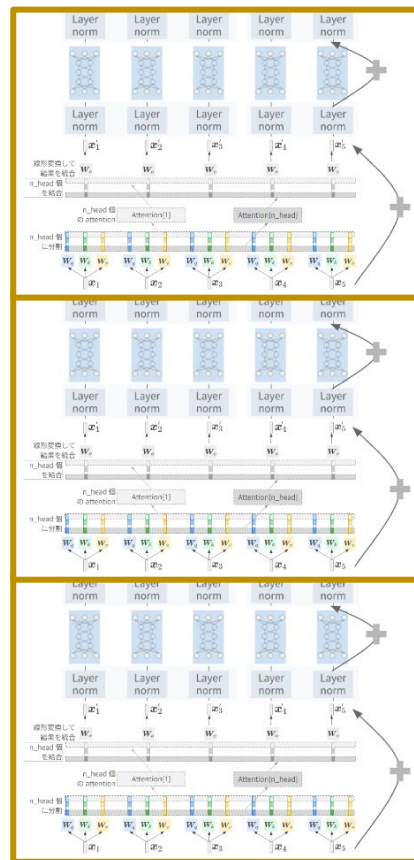


エンコーダ⑧：繰り返し

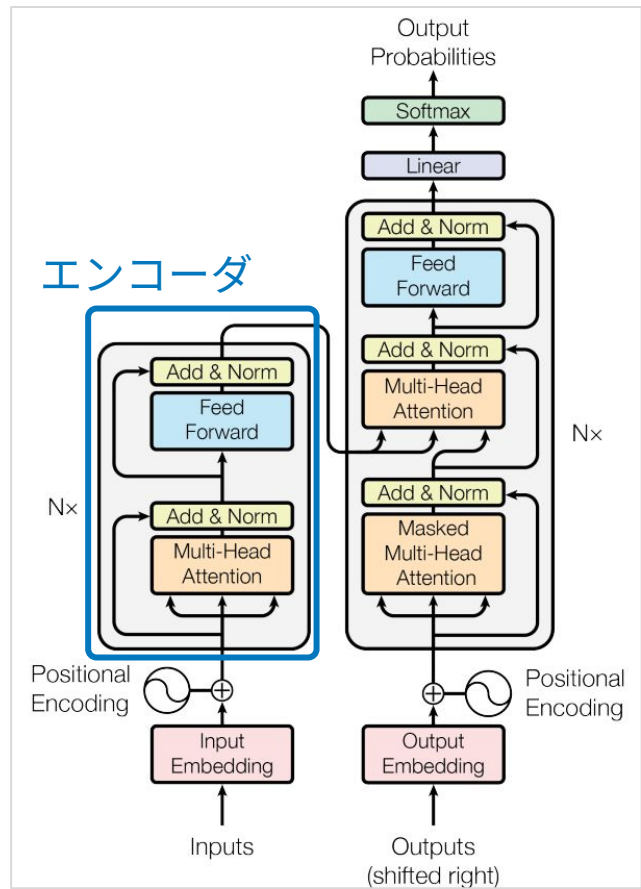
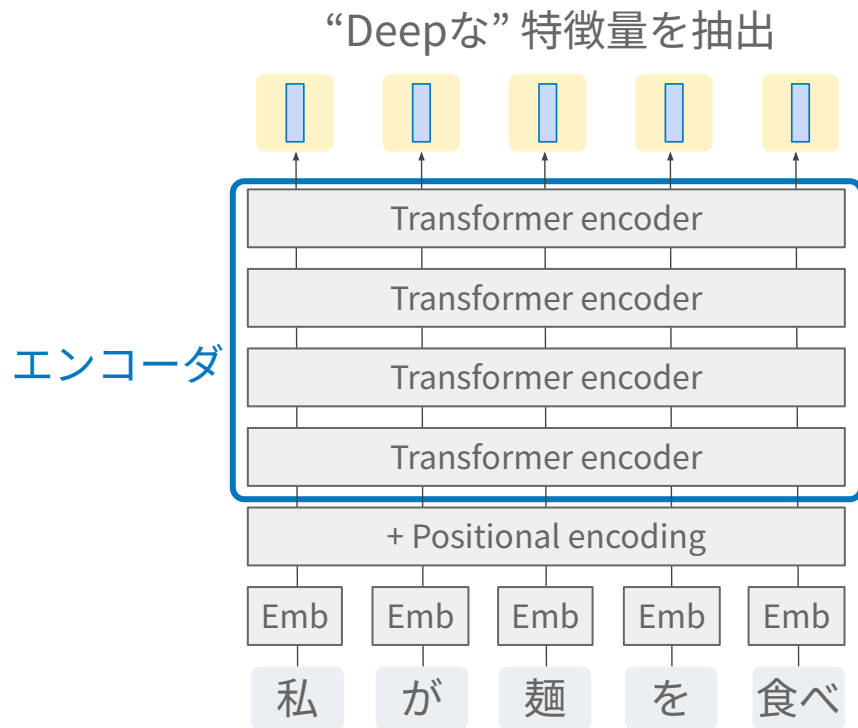
Transformer encoder

Transformer encoder

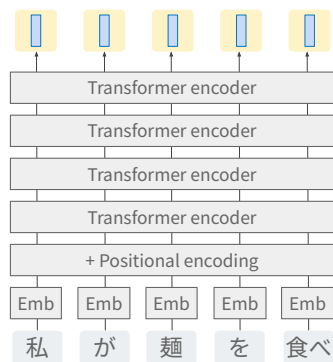
Transformer encoder



エンコーダまとめ



デコーダ①：単語埋め込み & positional encoding



位置を回転位置に変換したベクトル

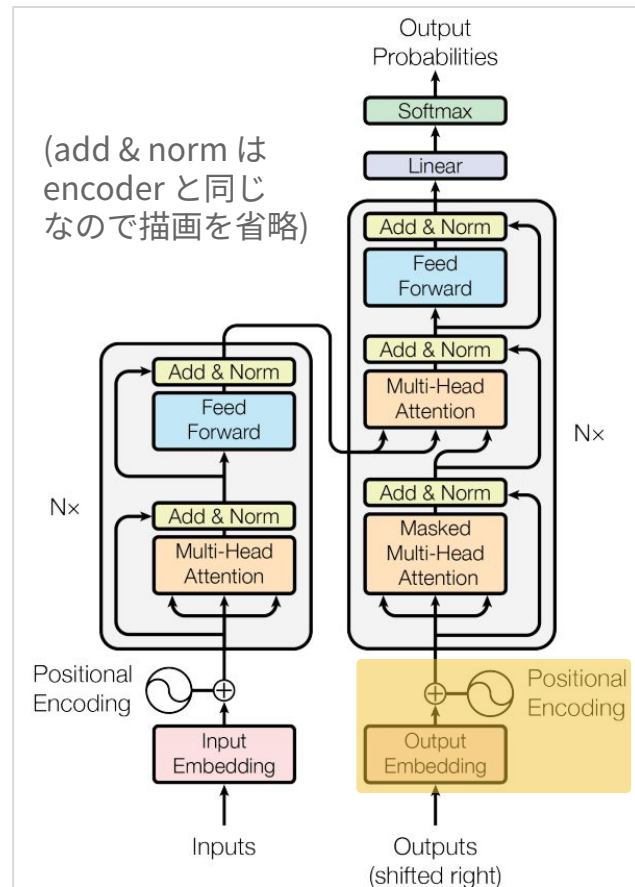
$$t_{id} = \begin{cases} \sin\left(\frac{i}{10000^{2d/4}}\right) \\ \cos\left(\frac{i}{10000^{2d/4}}\right) \end{cases} + \sin\left(2\pi \frac{i}{f}\right)$$

i は位置 index, d は次元 index f は回転周期, d が増大 $\rightarrow f$ が増大

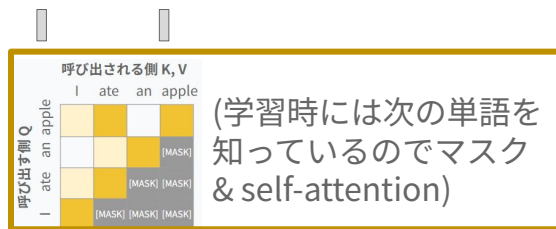
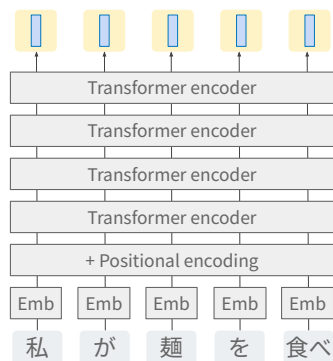
回転位置

A diagram showing the positional encoding for the input tokens. The tokens are 私 (I) and eat. Each token is passed through an embedding layer (Emb). The resulting embeddings are then added to a positional encoding vector (represented by a blue vertical bar) to produce the final input embeddings.

I eat これまでに予測した単語



デコーダ②：masked multi-head self-attention

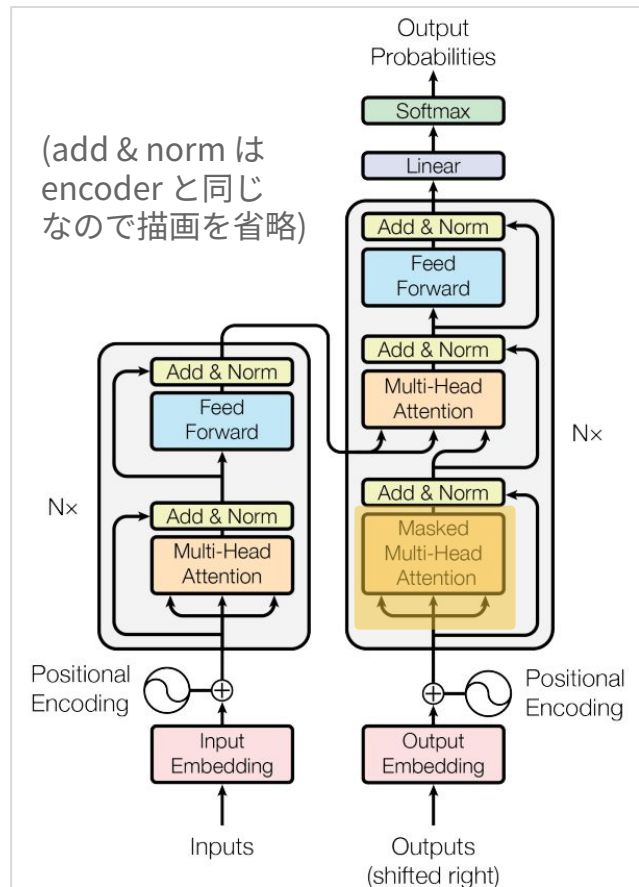


位置を回転位置に変換したベクトル

$$t_{id} = \begin{cases} \sin\left(\frac{i}{10000^{2d/4}}\right) \\ \cos\left(\frac{i}{10000^{2d/4}}\right) \end{cases} + \begin{cases} \sin\left(2\pi \frac{i}{T}\right) \\ \cos\left(2\pi \frac{i}{T}\right) \end{cases}$$

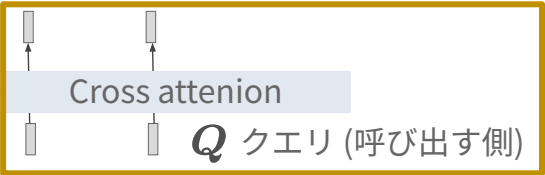
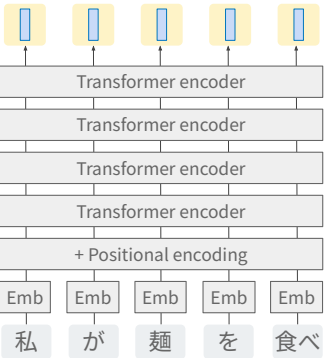
i は位置 index, d は次元 index

回転位置
回転周期, d が増大 $\rightarrow T$ が増大



デコーダ③：cross-attention で特徴を選択的に抽出

バリュー・キー
(呼び出される側。
各クエリに必要な特徴を返す)



位置を回転位置に変換したベクトル

$$t_{id} = \begin{cases} \sin\left(\frac{i}{10000^{2d/4}}\right) \\ \cos\left(\frac{i}{10000^{2d/4}}\right) \end{cases} + \begin{cases} \sin\left(2\pi \frac{i}{T}\right) \\ \cos\left(2\pi \frac{i}{T}\right) \end{cases}$$

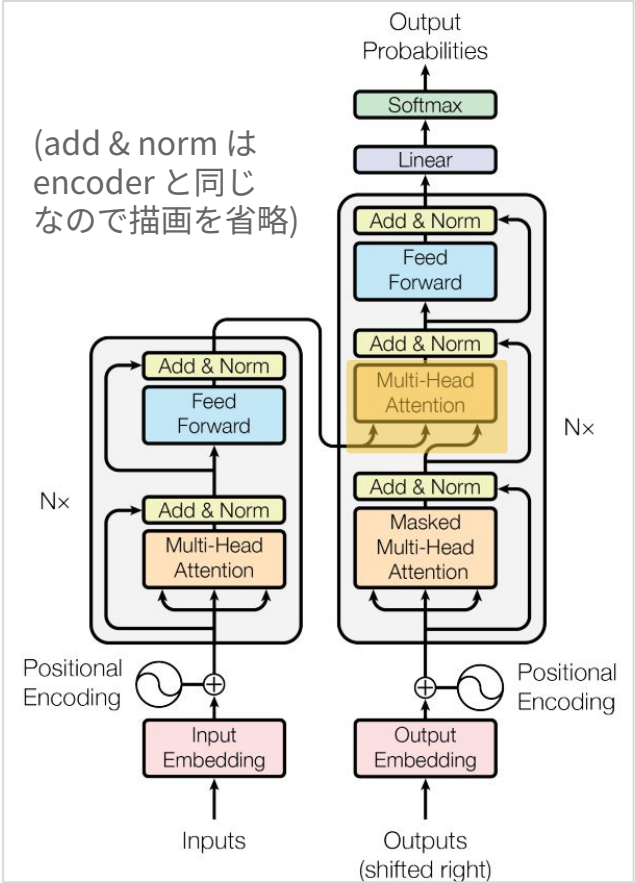
i は位置 index, d は次元 index

回転位置
回転周期: d が増大 → T が増大

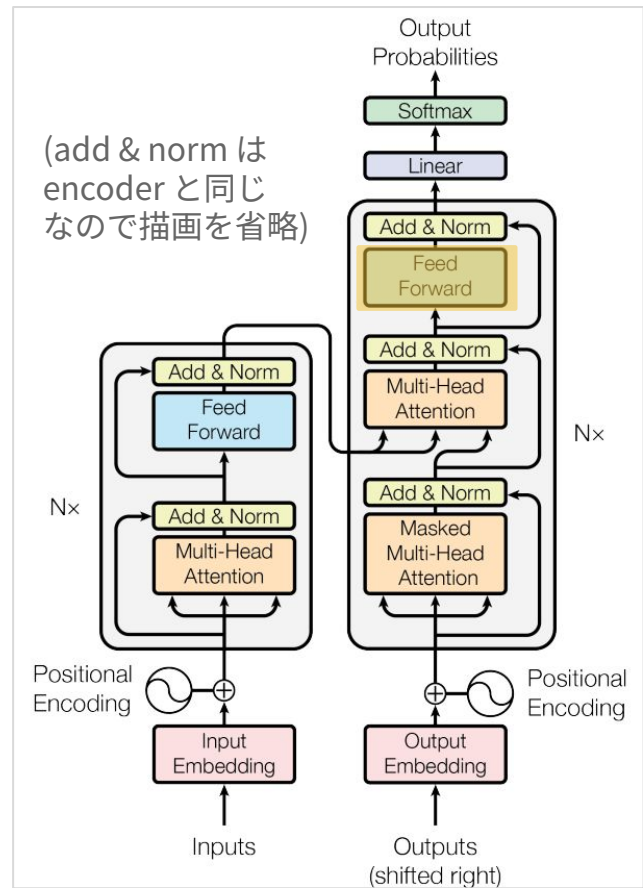
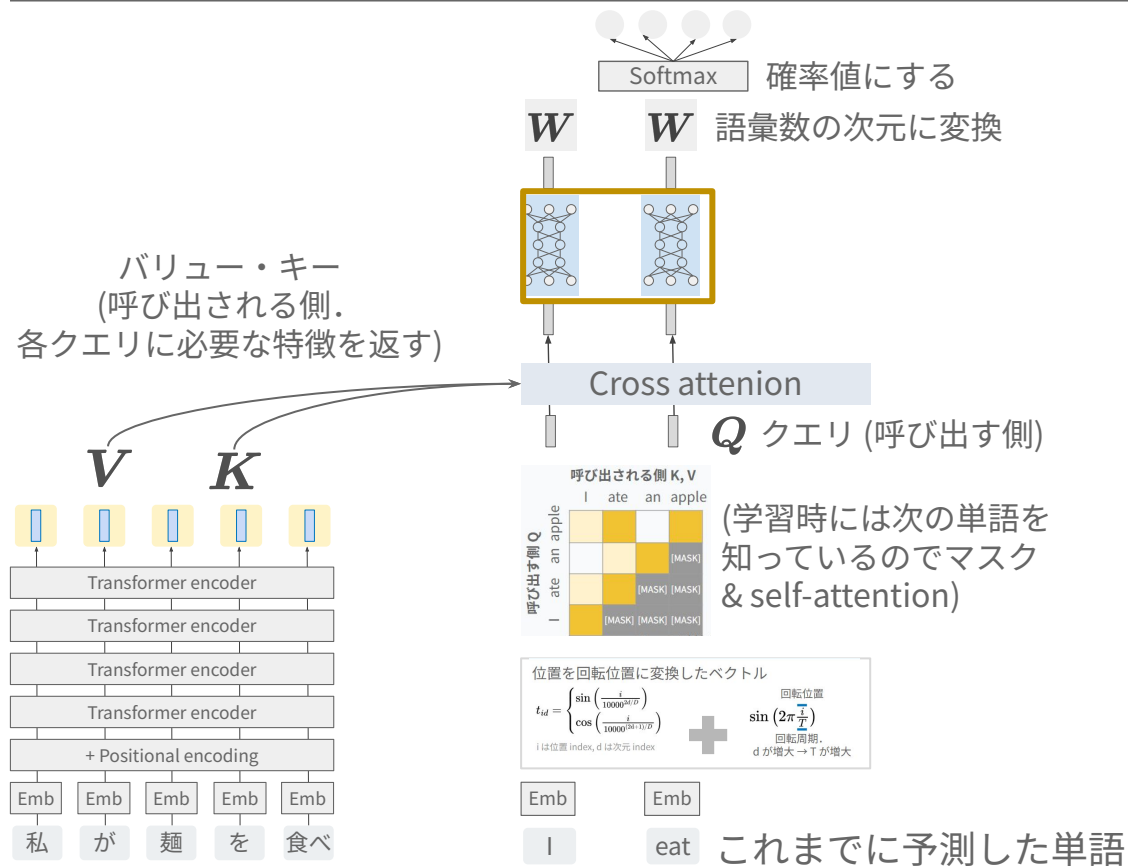
Emb

Emb

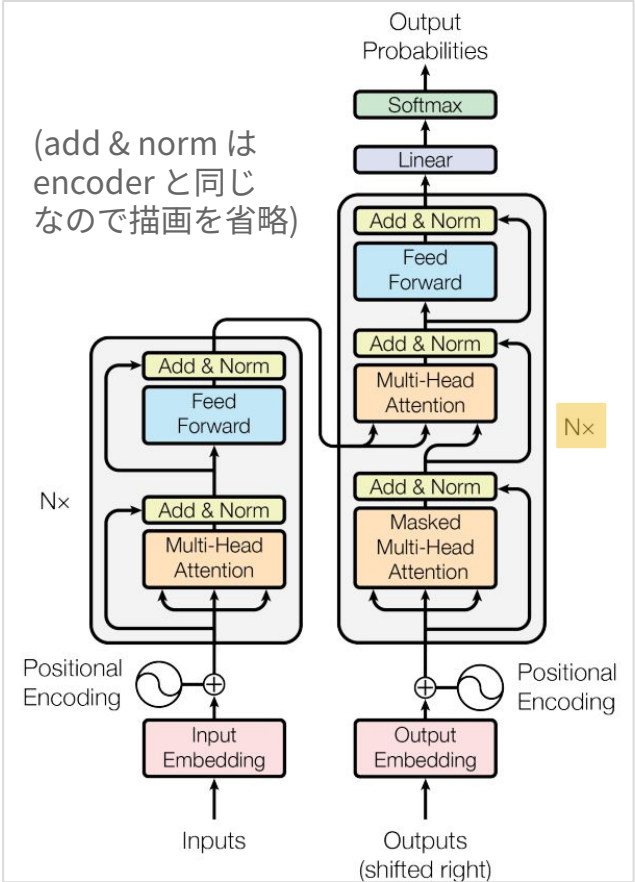
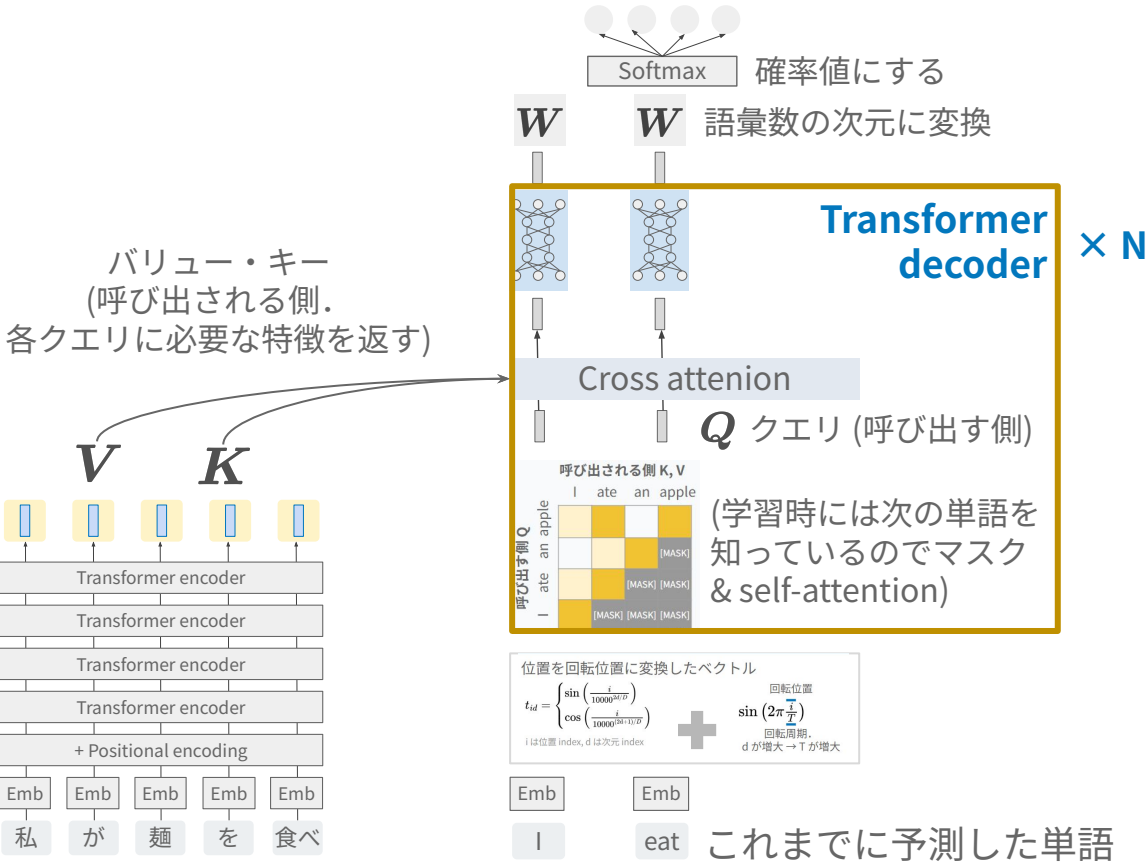
I eat これまでに予測した単語



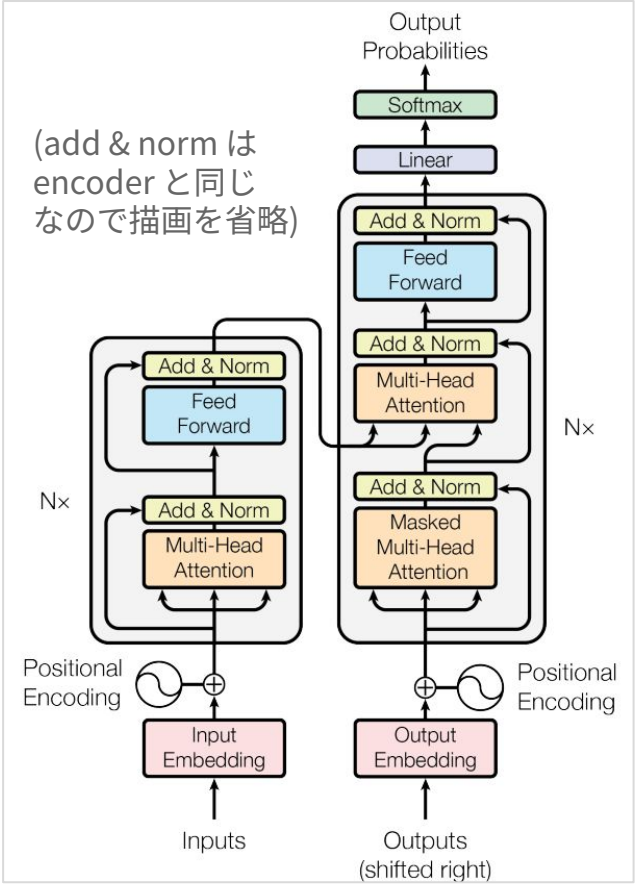
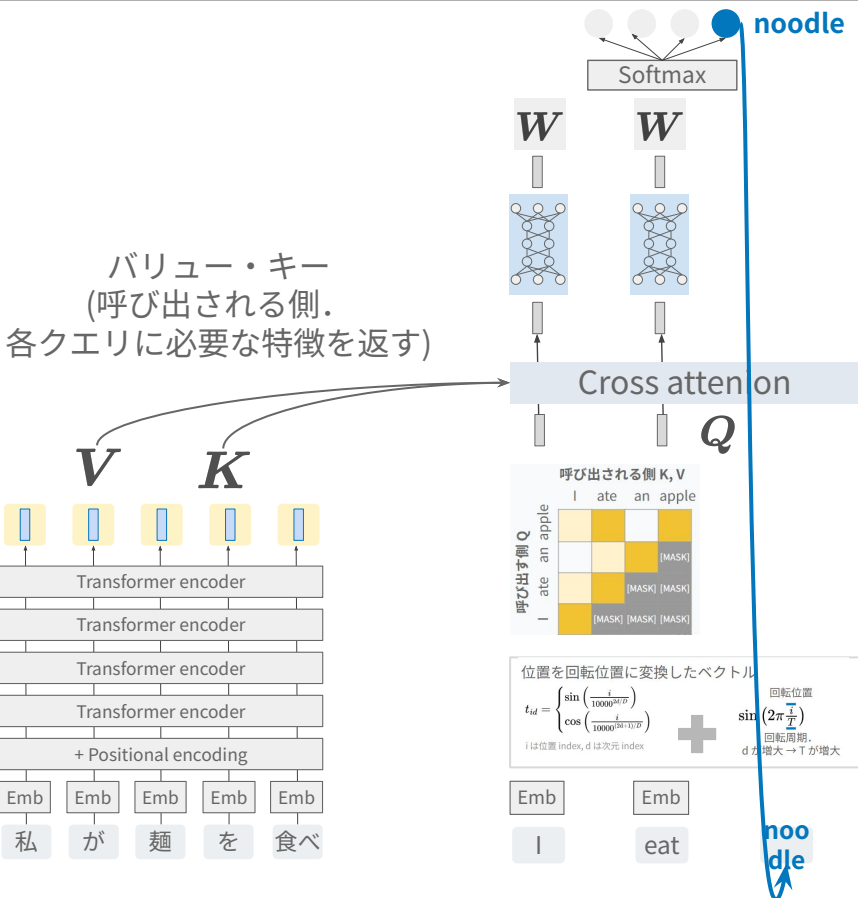
デコーダ④：feed-forward



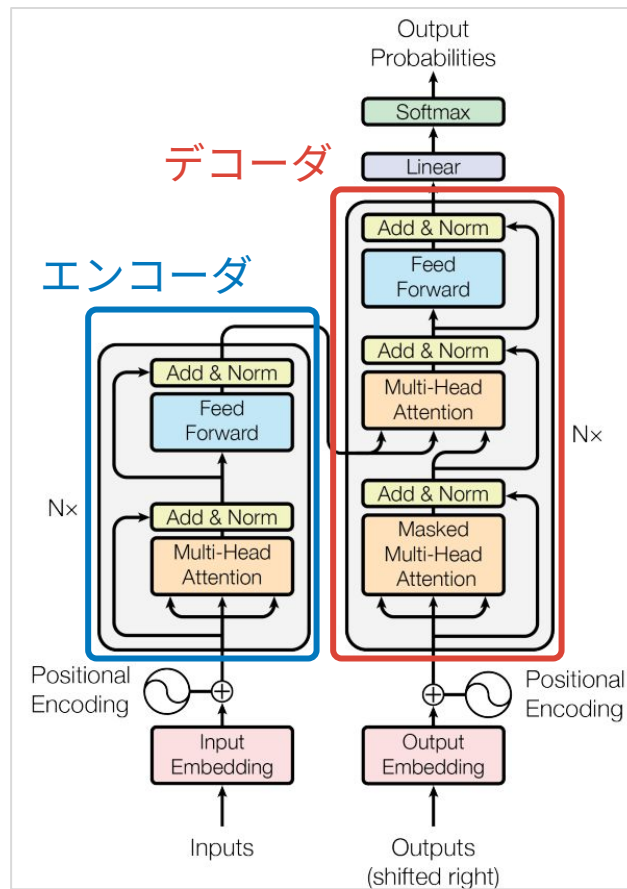
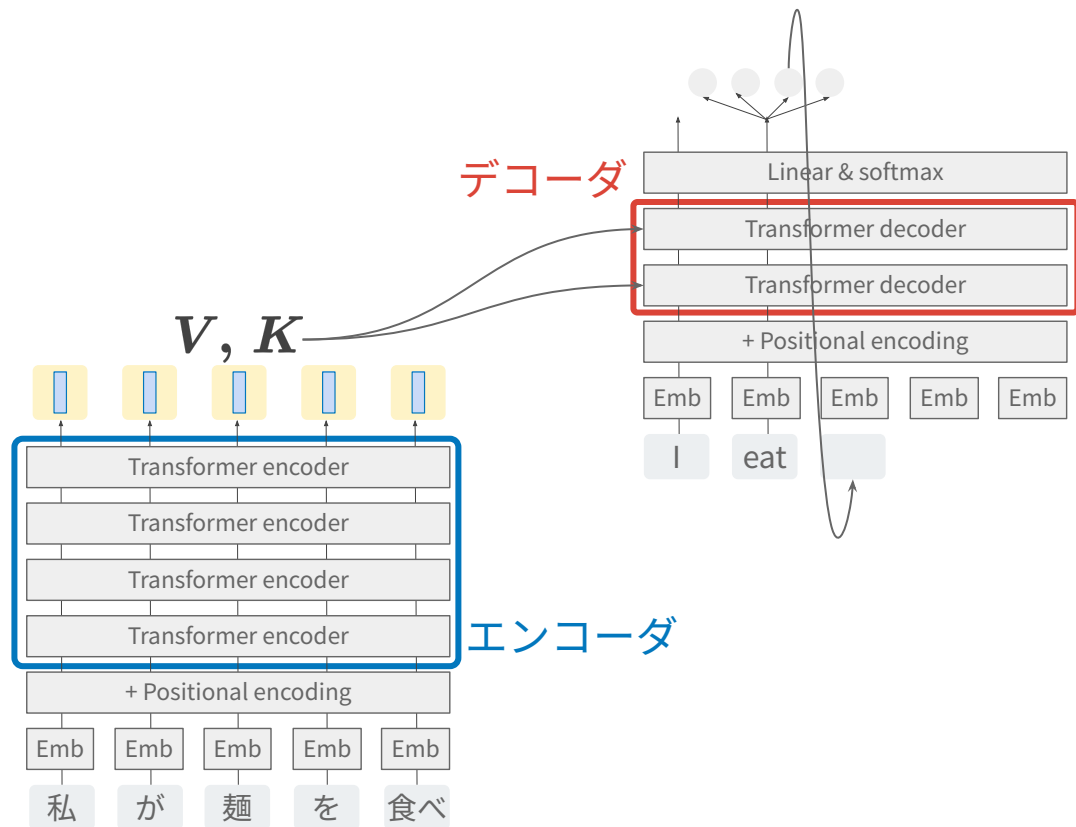
デコーダ⑤：繰り返し



デコーダ⑦：選んだ単語はデコーダの入力に戻す



エンコーダ&デコーダまとめ



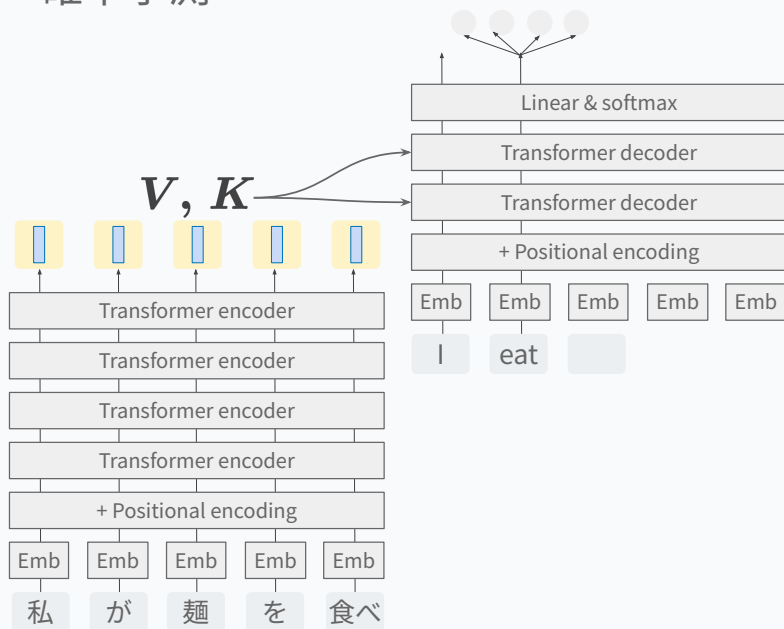
Transformer から Generative Pre-Trained Transformer (GPT) へ

Radford et al., “Improving Language Understanding by Generative Pre-Training,” 2018. (GPT-1)

Transformer から GPT へ

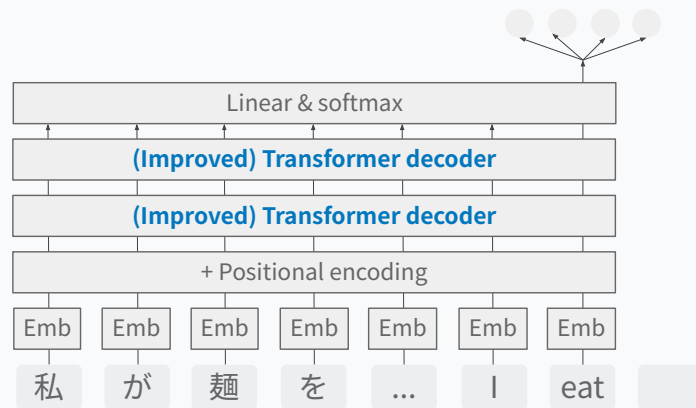
Transformer (encoder-decoder)

エンコーダで特徴抽出．デコーダで確率予測

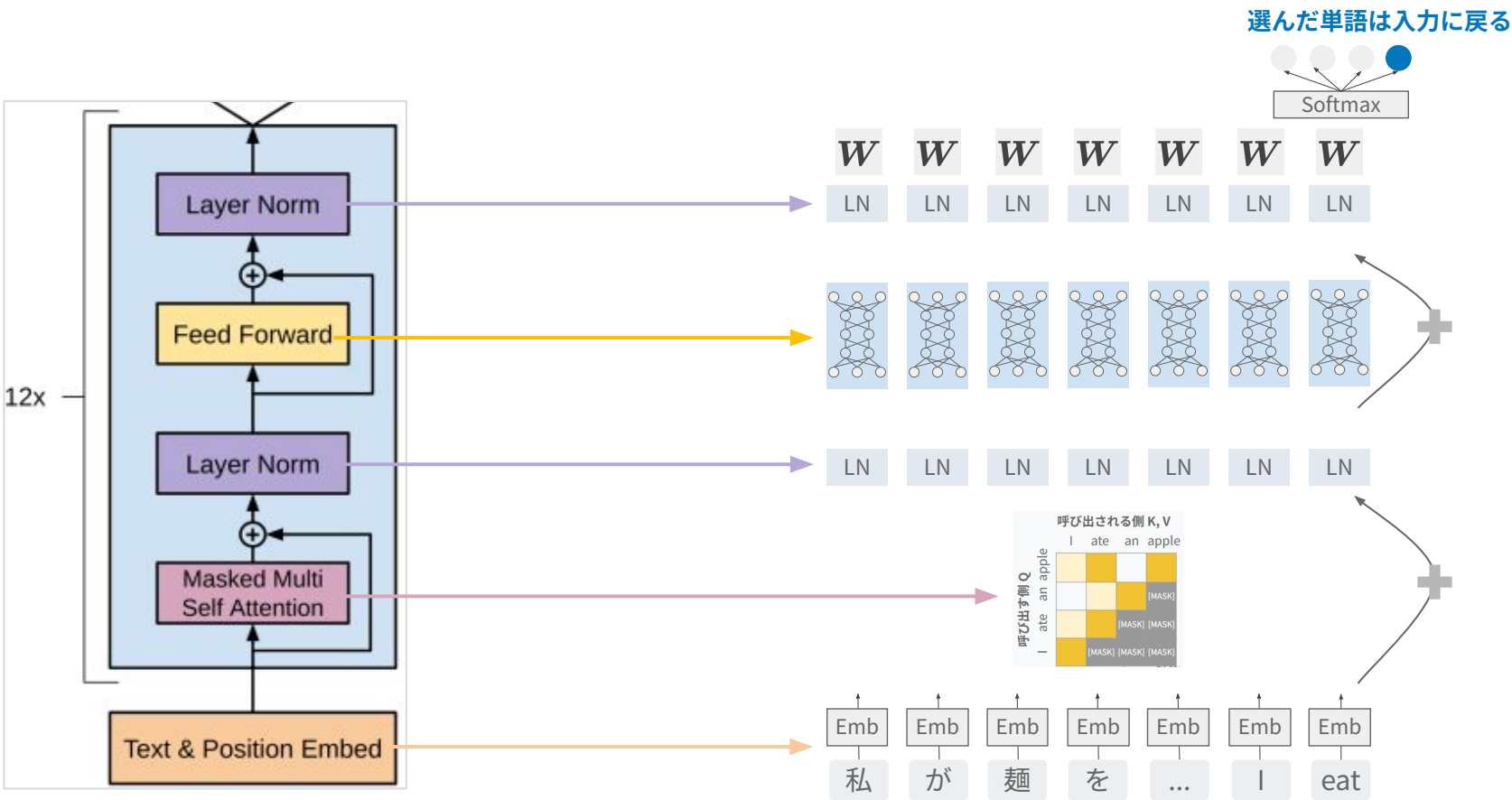


GPT (decoder-only)

特徴抽出も確率予測もデコーダで．
基本は Transformer decoder と同じ
だが，いくつかの改良が入っている



GPT-1 architecture



まとめ

まとめ

- Transformer を理解しよう
 - multi-head attention
 - scaled dot-product attention
 - masked attention
 - residual connection
 - layer normalization
 - (sinusoidal) positional encoding
- GPT を理解しよう
 - Decoder-only model でシンプルに

任意の課題 (提出は不要．採点対象にはしない．)

いずれかのコードを解読すること．今日の内容を理解できればいける．

Transformer (encoder-decoder)

<https://github.com/harvardnlp/annotated-transformer/blob/master/AnnotatedTransformer.ipynb>

The Annotated Transformer

[Attention is All You Need](#)

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
---	---	--	--

Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Lukasz Kaiser* Google Brain lukaszkaiser@google.com
--	---	--

Illia Polosukhin* †
illia.polosukhin@gmail.com

- v2022: Austin Huang, Suraj Subramanian, Jonathan Sum, Khalid Almubarak, and Stella Biderman.
- [Original: Sasha Rush.](#)

The Transformer has been on a lot of people's minds over the last year five years. This post presents an annotated version of the paper in the form of It reorders and deletes some sections from the original paper and adds comments throughout. This document itself is a working notebook, and should implementation. Code is available [here](#).

Table of Contents

- [Prelims](#)
- [Background](#)
- [Part 1: Model Architecture](#)
- [Model Architecture](#)

GPT (decoder-only)

<https://github.com/karpathy/minGPT>

README MIT license

minGPT

available GPT implementations



minGPT



A PyTorch re-implementation of [GPT](#), both training and inference. minGPT tries to be small, clean, interpretable and educational, as most of the currently available GPT model implementations can be a bit sprawling. GPT is not a complicated model and this implementation is appropriately about 300 lines of code (see [mingpt/model.py](#)). All that's going on is that a sequence of indices feeds into a [Transformer](#), and a probability distribution over the next index in the sequence comes out. The majority of the complexity is just being clever with batching (both across examples and over sequence length) for efficiency.